Plug-in CaWaQS-Viz 004.06

Visualizer for the modelling platform CaWaQS 3.57



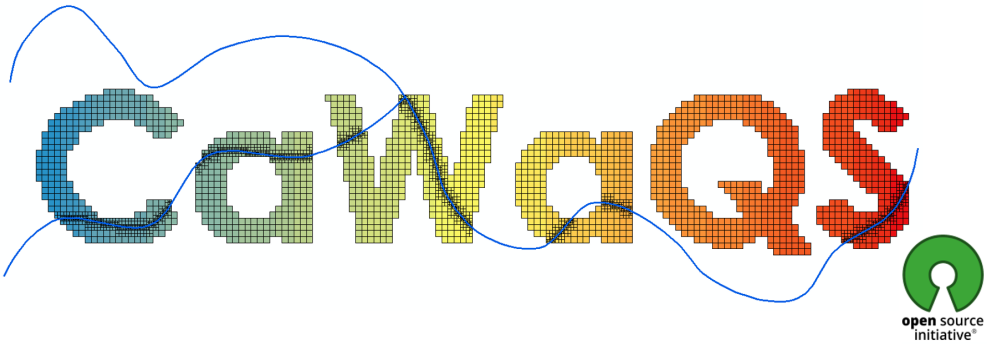
User Guide

Nicolas FLIPO

Simone MAZZARELLI

Nicolas GALLOIS

Lise-Marie GIROD



November 10, 2026

Mines Paris/ARMINES, PSL Université - Centre de Géosciences
35, rue Saint-Honoré
F-77 305 FONTAINEBLEAU Cedex

Contents

Introduction vi

What's New vii

I User Guide 1

1 Prerequisites and Installation 2
- 1.1 Option 1 — Pixi Environment (recommended) . . . 3
  - 1.1.1 Install `pixi` (once per machine) . . . 3
  - 1.1.2 Clone the repository, initialise the backend, and install the environment . . . 3
  - 1.1.3 Launch QGIS . . . 4
  - 1.1.4 Updating the plug-in . . . 4
- 1.2 Option 2 — QGIS Plugin Manager via ZIP . . . 4
  - 1.2.1 Build a clean install ZIP . . . 5
  - 1.2.2 Install the ZIP in QGIS . . . 5
  - 1.2.3 What happens on first launch . . . 7
- 1.3 Advanced Installation Settings (DEV) . . . 7
  - 1.3.1 Dependency Installation . . . 7
  - 1.3.2 Developer Installation . . . 8

2 Configuration 9
- 2.1 Overview . . . 10
  - 2.1.1 Mode of CaWaQS-Viz Configuration . . . 11
- 2.2 Automatic Detection of Project Settings . . . 11
  - 2.2.1 Probe 1 — from the CaWaQS output directory . . . 12
  - 2.2.2 Probe 2 — from the QGIS project neighbours . . . 13
- 2.3 Project Configuration from a `.json` File . . . 13

3 CaWaQS-Viz Features **15**
3.1 "`Calibration Tools`": Calibration Tools . . . 16
3.1.1 WIP: Modifying Parameterisation Files . . . 17
3.1.2 WIP: Linking an Attribute Table to its Resolution . . . 18
3.1.3 `Statistical Criteria`: Calculation of Statistical Performance Criteria at Observation Points . . . 19
3.1.3.1 Statistical criteria calculated by the "`Statistical criteria`" dialog box . . . 19
3.1.3.2 Output of the "`Statistical Criteria`" dialog box . . . 19
3.1.4 WIP `Run-off coefficient calculation`: Calculation of the Run-off Coefficient at Flow Measurement Stations . . . 21
3.2 `Compare sim/obs`: Comparison of Observed and Simulated Time Series . . . 23
3.3 `Spatial Map`: Spatial Mapping of Hydrological Variables . . . 24
3.4 `Budget Bar Plot`: Hydrological Budget Visualization . . . 25
3.5 `Hydological Regime`: Hydrological Regime Visualization . . . 26
3.6 `Extract Data from a sub area`: Extraction of Results in a Polygon . . . 27
3.6.1 Parameters . . . 27
3.6.2 Outputs . . . 27
3.6.3 Units . . . 28
3.6.4 Parameters — Internal Values . . . 28
3.6.4.1 The internal-values GeoPackage . . . 28
3.6.5 Parameters — Boundary Values . . . 29
3.6.5.1 Unit and sign convention . . . 29
3.6.5.2 Default mode: the face-flux CSV . . . 29
3.6.5.3 GeoPackage mode: the AQ-boundary GeoPackage . . . 29
3.6.6 Using the extraction from a notebook . . . 30
3.6.6.1 Internal values: the `specs` tuples . . . 30
3.6.6.2 Boundary values: `mask_aq_boundary` . . . 32

4 Using CaWaQS-Viz from a Notebook (without QGIS) **33**
4.1 Using CaWaQS-Viz from a Notebook (without QGIS) - Prerequisites . . . 34
4.2 Setting up the environment . . . 34
4.3 Running on a remote server . . . 34
4.4 Choosing a template . . . 35
4.5 Workflow . . . 35

5 Manual Project Configuration (Advanced) **36**
5.1 Contents of the QGIS Project Used for Processing CaWaQS Simulation Results . . . 37
5.2 Configuring the Project Geometry from an Existing QGIS Project . . . 39
5.3 Configuring Resolutions from a `.json` Configuration File . . . 42

II Developer Guide **44**

6 Key Concepts for Developers **45**
6.1 Coupling Observation Points to Mesh Cells . . . 46
6.2 The HYD Compartment: `ID_ABS` vs. `ID_GIS` . . . 46

7 Architecture **48**
7.1 Split Architecture: HTAS and CWV . . . 49
7.2 The Three Layers of the Backend . . . 49
7.3 Example Workflow: the Budget Bar Plot . . . 52
7.4 Entry Gates . . . 53
7.4.1 Client gate — L1 - HT CLIENT - MACRO . . . 53
7.4.2 Developer gate — L2 - HT DEVELOPER - MICRO . . . 53

8 Development Workflow **54**
8.1 Adding a New Dialog Operation . . . 54
8.2 When Editing — Checklist . . . 54
8.3 Conventions . . . 55
8.4 Version Update . . . 55
8.5 Code Documentation . . . 56

9 Developer Tooling **58**
9.1 Debugging Tools . . . 59
9.1.1 The `Plugin Reloader` plugin . . . 59
9.1.2 The QGIS plugin `First Aid` . . . 59
9.1.3 VS-Code . . . 60
9.2 Creating Dialog Boxes with `Qt Designer` . . . 64

Bibliography **65**

Introduction

The visualization tool CaWaQS-Viz was developed with the aim of visualizing the modelling outputs of the CaWaQS 3.57 modelling platform. This extension currently operates as a QGIS plugin under QGIS version 3.34and the same functionalities are as well accessible via API through jupyter notebook .

CaWaQS-Viz is structured around various tools and features that assist in the calibration process of a CaWaQS application and facilitate the visualization of time series and the spatial distribution of simulated hydrological variables.

CaWaQS-Viz is built on top of a standalone scientific backend HTAS (HydrologicalTwinAlphaSeries) and consumes it through a well-defined public interface.

Fig. 1 situates the plug-in within the wider CaWaQS toolchain.

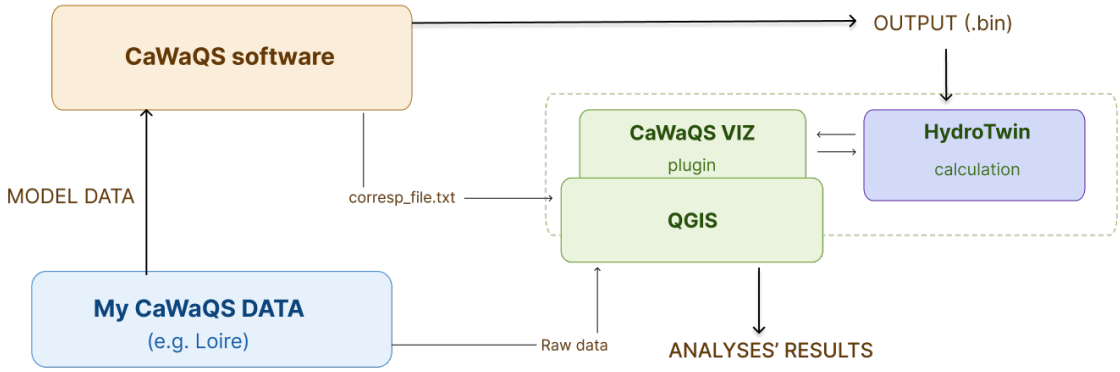


Figure 1: The CaWaQS-Viz ecosystem — position of the plug-in within the CaWaQS toolchain.

What's New

Below is the list of the latest bug fixes and feature implementations in the CaWaQS-Viz application.

Type/Tag	Description	Section
New feature	Isolated backden and frentend in 2 different repositories connected by APIs. The backend is the first version of the Hydrological Twin: Hydrological Twin Alpha Series (HTAS)	Sec. 2.1
New feature	Automated the configuration setting: only CaWaQS output repository needed	Sec. 5.2
New feature	Mask extraction. Clipping results from a subarea starting from a shapefile	Sec. 3.6

Table 1: Latest implementations in the application

Latest stable tags:

CWV_alpha_3.72 tag: Link (05 June 2026) : Stable tag of CaWaQS-Viz coupled with HTAS.

Part I

User Guide

CHAPTER 1

Prerequisites and Installation

Contents

1.1 Option 1 — Pixi Environment (recommended) . . . 3
1.1.1 Install `pixi` (once per machine) . . . 3
1.1.2 Clone the repository, initialise the backend, and install the environment . . . 3
1.1.3 Launch QGIS . . . 4
1.1.4 Updating the plug-in . . . 4
1.2 Option 2 — QGIS Plugin Manager via ZIP . . . 4
1.2.1 Build a clean install ZIP . . . 5
1.2.2 Install the ZIP in QGIS . . . 5
1.2.3 What happens on first launch . . . 7
1.3 Advanced Installation Settings (DEV) . . . 7
1.3.1 Dependency Installation . . . 7
1.3.2 Developer Installation . . . 8

The CaWaQS-Viz plug-in was designed to run within the QGIS Geographic Information System (GIS). It was initially developed under QGIS 3.28[1], and is currently used under version 3.36. Correct operation is not guaranteed for use under later versions.

CaWaQS-Viz relies on a scientific backend, `HydrologicalTwinAlphaSeries` (HTAS), vendored in the repository as a GitLab submodule under `external/HydrologicalTwinAlphaSeries`. This submodule must be populated for the plug-in to run, whichever installation option is chosen.

Two installation options are available:

- **Option 1 — Pixi environment** (Sec. 1.1): installs QGIS and all dependencies inside the project folder in a reproducible, self-contained way. Recommended for developers and for reproducible deployments.
- **Option 2 — QGIS Plugin Manager via ZIP** (Sec. 1.2): the classic plug-in archive installed from within QGIS. More convenient for end users, but more fragile for dependency handling.

1.1 Option 1 — Pixi Environment (recommended)

This option installs QGIS and all dependencies inside the project folder *via* `pixi`. It provides a reproducible, self-contained environment in which QGIS, Python, and every dependency are pinned.

1.1.1 Install `pixi` (once per machine)

Here reported the information from the Pixi Installation page :

On macOS or Linux:

```
curl -fsSL https://pixi.sh/install.sh | sh
source $HOME/.bashrc
```

If your system does not have `curl`, use `wget`:

```
wget -qO- https://pixi.sh/install.sh | sh
```

On Windows:

```
powershell -ExecutionPolicy Bypass -c "irm -useb https://pixi.sh/install.ps1 | iex"
```

1.1.2 Clone the repository, initialise the backend, and install the environment

```
mkdir $HOME/.plugins-qgis
cd $HOME/.plugins-qgis
git clone https://gitlab.com/gviz/cawaqsviz.git
```

[1] Available at: https://www.qgis.org/fr/site/forusers/download.html

```
cd cawaqsviz
git submodule update --init external/HydrologicalTwinAlphaSeries
pixi install
```

From the **cawaqsviz** repository root, initialise only **external/HydrologicalTwinAlphaSeries**. Do not use a recursive submodule update unless you explicitly need the internal backend documentation sources and have access to their private repositories.

1.1.3 Launch QGIS

```
pixi run cawaqsviz
```

The last step is to activate the plug-in in QGIS from the main menu: "**Plugins**" → "**Manage and Install Plugins**", then select "**Installed**" in the navigation window that opens (Fig. 1.2).

1.1.4 Updating the plug-in

Once installed, go to the **cawaqsviz** directory, optionally update to the latest version, and run with **pixi**:

```
cd $HOME/.plugins-qgis/cawaqsviz
git pull                                                    # update cawaqsviz
    itself; may move the pinned backend SHA
git submodule sync -- external/HydrologicalTwinAlphaSeries  # refresh the backend
    remote URL if it changed upstream
git submodule update --init external/HydrologicalTwinAlphaSeries  # check the backend out
    to the SHA pinned by cawaqsviz
pixi run cawaqsviz                                          # launch QGIS from the
    pixi environment
```

1.2 Option 2 — QGIS Plugin Manager via ZIP

The CaWaQS-Viz visualisation tool has been developed and tested on QGIS 3.28 and later. WARNING: this type of installation is more fragile than Option 1 for handling dependencies. The development team recommends Option 1 for source-based development and reproducible environments.

Do **not** use GitLab's "**Download ZIP**" button — its archive breaks the plug-in install. You must build the ZIP yourself from a proper **git clone** with submodule contents, as shown below.

The GitLab archive download fails for two reasons:

- the top-level folder is named after the branch or tag (e.g. `cawaqsviz-main-abc1234/`), but QGIS requires it to be named exactly `cawaqsviz`. Installing the GitLab ZIP directly produces an error such as `No module named 'cawaqsviz-<branch>';`
- the GitLab submodule contents are not included — only a pointer. The vendored `external/HydrologicalTwinAlp` folder will be empty and the first-launch bootstrap will fail with a "submodule missing" error.

1.2.1 Build a clean install ZIP

On Linux/macOS:

```
# 1. Clone into a temporary location with submodule contents
cd /tmp
git clone https://gitlab.com/gviz/cawaqsviz.git
cd cawaqsviz
git submodule update --init external/HydrologicalTwinAlphaSeries

# 2. Build the ZIP from the parent directory, excluding (optionally) dev-only artefacts
cd ..
zip -r cawaqsviz.zip cawaqsviz \
    -x "cawaqsviz/.git/*" \
    -x "cawaqsviz/.pixi/*" \
    -x "cawaqsviz/__pycache__/*" \
    -x "cawaqsviz/**/__pycache__/*"
```

On Windows (PowerShell), use Git for Windows to clone (the same `git` commands as above), then build the ZIP by right-clicking the `cawaqsviz/` folder (→ "`Send to`" → "`Compressed (zipped) folder`"), or with PowerShell:

```
Compress-Archive -Path cawaqsviz -DestinationPath cawaqsviz.zip
```

The resulting `cawaqsviz.zip` has the correct top-level folder name and includes the backend submodule contents.

1.2.2 Install the ZIP in QGIS

In the QGIS top menu bar, click "`Plugins`" then "`Manage and Install Plugins`". The plugin management and import window opens. Select "`Install from ZIP`" (Fig. 1.1) and choose `cawaqsviz.zip`, then click "`Install plugin`".

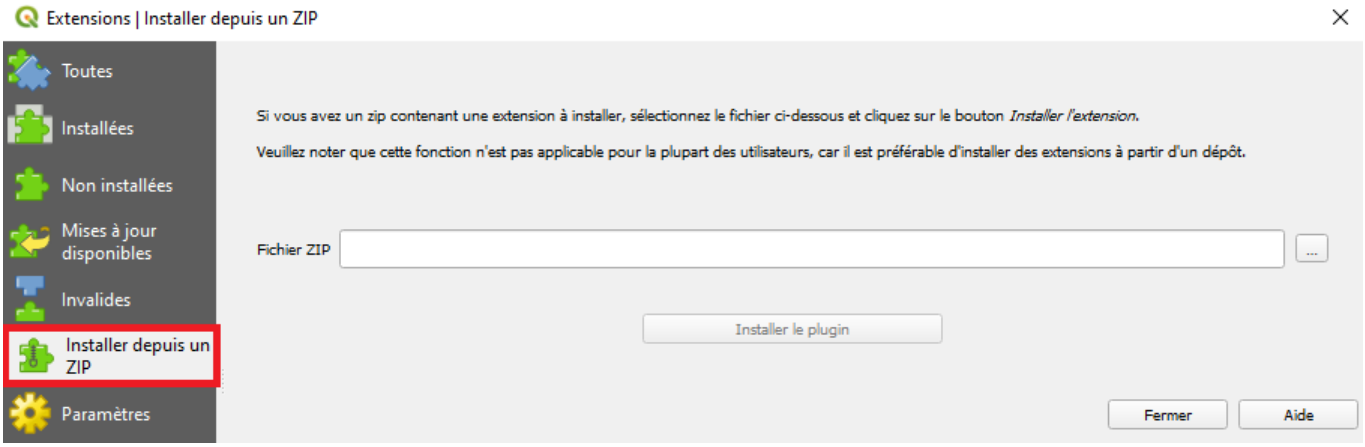


Figure 1.1: QGIS dialog box for plugin management.

Once the `.zip` file has been extracted in QGIS, it may be necessary to activate it in the same plugin management window by navigating to the "`Installed`" panel. The CaWaQS-Viz extension must be checked to appear in the QGIS toolbar (Fig. 1.2).

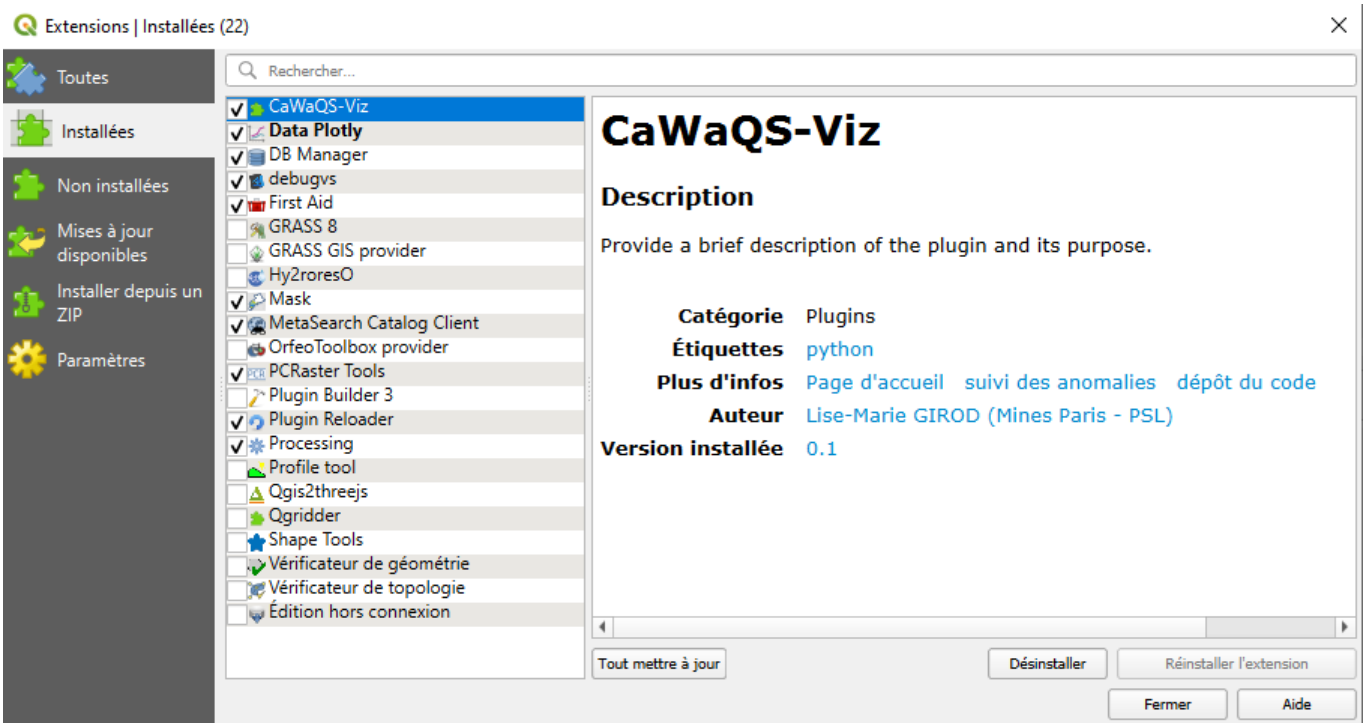


Figure 1.2: Activating the CaWaQS-Viz extension in the QGIS application.

Once the application is present in the QGIS toolbar, it is ready to be launched with a single click on the icon .

1.2.3 What happens on first launch

On the first launch after a ZIP installation, the plug-in automatically runs `pip install -user -upgrade external/HydrologicalTwinAlphaSeries` to register the backend and resolve its Python dependencies. This requires an internet connection and lands in your per-user site-packages (`~/.local/lib/pythonX.Y/site-package` on Linux, `%APPDATA%\Python\...` on Windows). Subsequent launches are no-ops.

On Debian/Ubuntu and derivatives that ship a PEP 668 "externally-managed" system Python, the plug-in automatically retries with `-break-system-packages` (the install still lands in your per-user site-packages, not in apt-managed system paths).

If the install fails, the QGIS Python console prints the exact `pip` command you can run manually to diagnose.



To use any new update of the plug-in installed via ZIP, it will be necessary to repeat this entire procedure (Sec. 1.2). With Option 1, updating is done with `git pull` instead (Sec. 1.1.4).

1.3 Advanced Installation Settings (DEV)

1.3.1 Dependency Installation

Dependencies are handled automatically. With Option 1, `pixi` pins and installs QGIS, Python, and every dependency inside the project folder (Sec. 1.1). With Option 2, the first-launch bootstrap resolves the backend and its Python dependencies (Sec. 1.2.3). No manual dependency installation is normally required.



If the first-launch bootstrap fails, the QGIS Python console prints the exact `pip` command to run. As a fallback, dependencies can still be installed manually from the Python interpreter built into QGIS. Installing them from any other interpreter will not allow QGIS to access the required libraries.

The core Python dependencies are: `pandas`, `numpy`, `ipython`, `matplotlib`, and `geopandas`. As a manual fallback, open a QGIS Python console[2] and run:

```
>>> import pip
>>> pip.main(['install', 'pandas', 'numpy', 'ipython', 'matplotlib', 'geopandas'])
```

[2] By clicking on  in the QGIS interface.

1.3.2 Developer Installation

For development purposes, and to make it easier to update the plug-in from the QGIS interface, the plug-in can be installed directly in the QGIS *native* extensions directory. The GitLab repository for CaWaQS-Viz can be cloned from a command terminal, remembering to initialise the backend submodule:

```
$ git clone https://gitlab.com/gviz/cawaqsviz
$ cd cawaqsviz
$ git submodule update --init external/HydrologicalTwinAlphaSeries
```

The application directory must be placed at the location (folder) of the native QGIS applications so that any modification of the plug-in source code can be taken into account by QGIS at startup. The QGIS directory is accessible *via* the following path:

```
#windows
C:/Programmes/QGIS3.X.X.X/apps/qgis-ltr/python/plugins/
#linux
~/.local/share/QGIS/QGIS3/profiles/default/python/plugins/
```

The plug-in code structure is described in section 7.



Autocompletion of features related to the qgis library is available provided the Python interpreter used by QGIS is used. A virtual environment can be initialised for this purpose.

N.B.: To facilitate plug-in development, the following utilities can be installed from the QGIS application manager:

1. **Reloader** allows reloading the plug-in without restarting QGIS after each code modification,
2. **FirstAid** provides a detailed view of errors returned by the QGIS Python interpreter as well as the various objects and variables related to the extension.

CHAPTER 2

Configuration

Contents

2.1 Overview **10**
2.1.1 Mode of CaWaQS-Viz Configuration 11
2.2 Automatic Detection of Project Settings **11**
2.2.1 Probe 1 — from the CaWaQS output directory 12
2.2.2 Probe 2 — from the QGIS project neighbours 13
2.3 Project Configuration from a `.json` File **13**

2.1 Overview

The CaWaQS-Viz extension integrates several features accessible from its home dialog box (Fig. 2.1):

- a dedicated entry for configuring the CaWaQS application ("`Settings`" - Sec. 2.1.1 to 2.3) ;
- calibration assistance tools ("`Calibration tools`" - Sec. 3.1) ;
- features for visualizing simulated and observed flow and piezometry time series ("`Compare Sim/Obs`" - Sec. 3.2) ;
- tools for visualizing the spatial distribution of hydrological components ("`Spatial Maps`" - Sec. 3.3) ;
- a hydrological budget visualization function ("`Budget Bar Plot`" - Sec. 3.4) ;
- a hydrological regime viewer ("`Hydrological regime`" - Sec. 3.5).

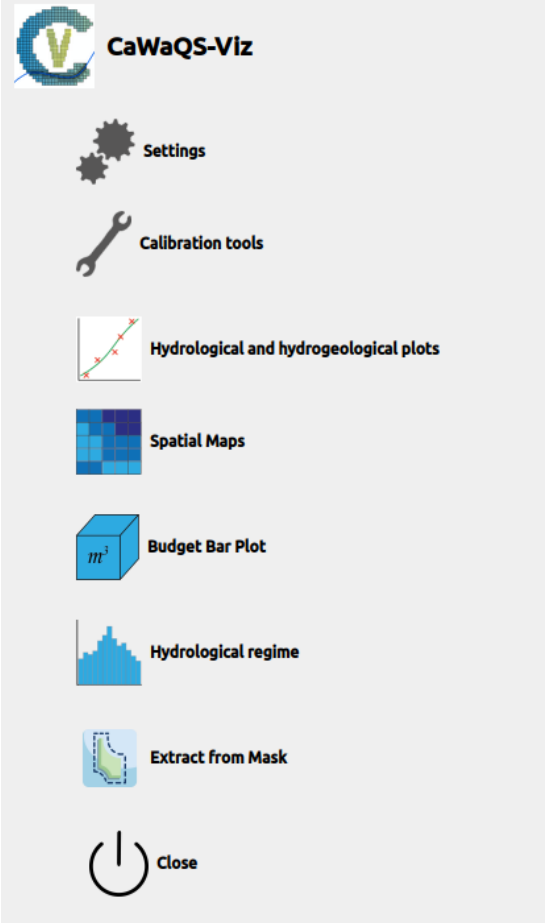


Figure 2.1: Home dialog box of the CaWaQS-Viz application.

The use of the CaWaQS-Viz extension is based on a pre-existing `GIS` project containing:

- the meshes of the various CaWaQS application modules, at the modelling output resolutions;
- observation points and/or extraction points, located within the bounds of the model extent.

2.1.1 Mode of CaWaQS-Viz Configuration

The plug-in must be configured to read the various modelling output files and/or CaWaQS input files as well as the various meshes. This parameterisation of the plug-in can be carried out in the "`Settings`" window. To do so, two procedures are available 2.2:

- FIRST TIME: configure the project manually in the second part of the dialog box (section 2.2).
- ALREADY SET: import a `.json` configuration file established beforehand (section 2.3).;

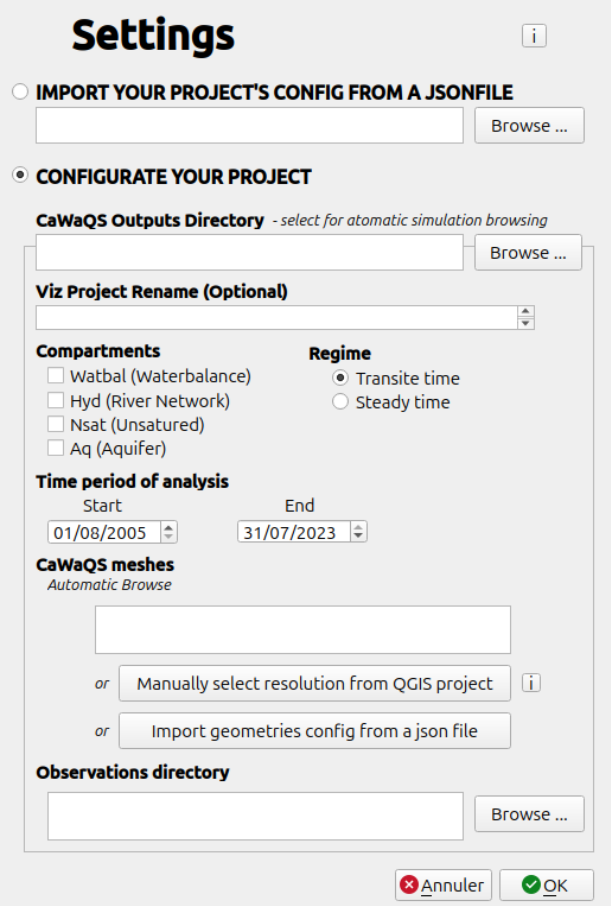


Figure 2.2: The "`Settings`" dialog box of the plug-in, selecte the creation of a new project.

The details of these two application configuration procedures are described in paragraphs 2.2 and 2.3 respectively.

2.2 Automatic Detection of Project Settings

When configuring a project manually (rather than importing a `.json` configuration file), CaWaQS-Viz can pre-fill most of the "`Settings`" dialog by inspecting the file system. The only action required from the user is to browse to the CaWaQS output directory: detection then runs automatically and fills in the compartments, the simulation years, the regime, the observation directory and the geometry configuration path.

Detection is a *best-effort* convenience. It relies entirely on the naming and layout conventions described below. If a convention is not met, the corresponding field is simply left empty and can be filled by hand — the only hard requirement is that the output directory contain at least one recognisable module sub-folder (see Sec. 2.2.1).

Detection is driven by two independent probes anchored on two different roots:

- the CaWaQS **output directory** you select (Sec. 2.2.1) — drives the compartments, years and regime;
- the **QGIS project file** (`.qgz` / `.qgs`) currently open (Sec. 2.2.2) — drives the geometry configuration, the observation directory and the project name.

2.2.1 Probe 1 — from the CaWaQS output directory

This probe is anchored on the output directory selected through the "`Browse`" button. It expects, directly inside that directory, one or more module output sub-folders, each named after its CaWaQS module:

Sub-folder	Detected compartment
`Output_AQ`	`AQ` (aquifers)
`Output_HYD`	`HYD` (hydraulic network)
`Output_WATBAL`	`WATBAL` (surface water balance)
`Output_NONSAT`	`NSAT` (unsaturated zone)

At least one of these sub-folders must be present; otherwise detection fails and the dialog must be filled manually.

Inside each module sub-folder, the **binary result filenames** determine the simulation years and the regime. Two filename patterns are recognised:

- `<MODULE>_<VAR>.YYYYYYYY.bin` — a **transient** yearly step. The eight digits encode two consecutive years, the start `YYYY` followed by the end `YYYY`, where the end is the start + 1 (e.g. `20102011`). Each such file contributes its start year; the earliest start and the latest end define the simulation range.
- `<MODULE>_<VAR>.00.bin` — a **steady-state** result. The presence of this pattern marks the run as steady.

The regime is resolved as follows: `Steady` if only steady filenames are found and no transient ones; otherwise `Transient`. If both patterns coexist, the run is classified as `Transient` and a warning is issued. If no year could be read from any filename, the range defaults to `1970–2022`.

The expected disposition therefore looks like:

```
<OUT_CAW>/
|-- Output_AQ/
|     |-- AQ_HEAD.20102011.bin      <-- transient (2010->2011)
|     |-- AQ_HEAD.20112012.bin      <-- transient (2011->2012)
|     +-- ...
|-- Output_HYD/
|     +-- HYD_Q.20102011.bin
|-- Output_WATBAL/
|     +-- WATBAL_...00.bin          <-- steady-state token
+-- Output_NONSAT/
      +-- ...
```

Code 2.1: Expected layout of the CaWaQS output directory.



The regime, the compartment list and the simulation years are read *only* from this probe. Browsing to the output directory is thus the single mandatory step for automatic detection.

2.2.2 Probe 2 — from the QGIS project neighbours

This probe is anchored on the currently open QGIS project file (`.qgz` / `.qgs`). It is purely best-effort: each of the three items below is looked up independently, and any of them may remain empty without preventing detection.

- **Geometry configuration** — the probe searches for files matching `config_geometries_*.json` in the *same folder* as the project file. If several match, the most recently modified one is used.
- **Observation directory** — the probe looks for a folder named `DATA_OBS` at the level of the project file, then one level up, then two levels up, and stops at the first match.
- **Project name** — derived from the folder that *contains* `DATA_OBS` when it is found; otherwise from the project file's parent (or grand-parent) folder name.

The expected neighbourhood of the project file is:

```
<project_root>/                      <-- name used as project name
|-- DATA_OBS/                        <-- observation directory
+-- <qgis_folder>/
      |-- my_project.qgz             <-- open QGIS project
      +-- config_geometries_*.json   <-- geometry config (newest wins)
```

Code 2.2: Expected layout around the QGIS project file.



Because the geometry configuration is searched only next to the `.qgz` / `.qgs` file, and `DATA_OBS` only up to two levels above it, a project saved outside its usual folder structure will leave these fields empty. This is expected — fill them manually via the "`Configure Geometry`" button and the observation browse field.

2.3 Project Configuration from a `.json` File

Once a project has been configured (whether by automatic detection or by hand), its configuration is written as `.json` files under `<OUT_CAW>/PostProcessing/CONFIG/`, namely `config_project_<name>.json` and `config_geometries_<name>.json`. These files can later be re-imported directly through the first configuration procedure, bypassing detection entirely.

Importing an external `.json` file can also be used to configure the project. This file is automatically saved into the Postprocessing directory of a project after its first configuration.

Upon execution of the CaWaQS-Viz application parameterisation dialog box, the project sub-directory is created in the specified post-processing directory. If the project does not yet exist in this directory, a sub-directory named after the project will be created, containing the project and resolution configuration files written in "**json**" format.

This project configuration **.json** file is structured as follows:

```
{
    "json_path_geometries": str,
    "projectName": str,
    "cawOutDirectory": str,
    "startSim": int,
    "endSim": int,
    "obsDirectory": str,
    "ppDirectory": str,
    "regime": str
}
```

The contents of each object in this **.json** file are as follows:

- "**json_path_geometries**": absolute path of the resolution configuration file;
- "**projectName**": project name;
- "**cawOutDirectory**": absolute path of the directory containing the CaWaQS modelling outputs;
- "**startSim**" and "**endSim**": simulation start and end years in **integer** format;
- "**obsDirectory**": absolute path of the parent directory containing the observed time series;
- "**ppDirectory**": absolute path of the post-processing directory. Note that the path must already exist. Otherwise, it will not be created by the application.
- "**regime**": modelling regime ("Transient" or "Steady")

CHAPTER 3

CaWaQS-Viz Features

Contents

3.1 `"Calibration Tools"`: **Calibration Tools** **16**
- 3.1.1 WIP: Modifying Parameterisation Files 17
- 3.1.2 WIP: Linking an Attribute Table to its Resolution 18
- 3.1.3 `Statistical Criteria`: Calculation of Statistical Performance Criteria at Observation Points 19
- 3.1.4 WIP `Run-off coefficient calculation`: Calculation of the Run-off Coefficient at Flow Measurement Stations 21

3.2 `Compare sim/obs`: **Comparison of Observed and Simulated Time Series** **23**

3.3 `Spatial Map`: **Spatial Mapping of Hydrological Variables** **24**

3.4 `Budget Bar Plot`: **Hydrological Budget Visualization** **25**

3.5 `Hydological Regime`: **Hydrological Regime Visualization** **26**

3.6 `Extract Data from a sub area`: **Extraction of Results in a Polygon** **27**
- 3.6.1 Parameters 27
- 3.6.2 Outputs 27
- 3.6.3 Units 28
- 3.6.4 Parameters — Internal Values 28
- 3.6.5 Parameters — Boundary Values 29
- 3.6.6 Using the extraction from a notebook 30

3.1 "Calibration Tools": Calibration Tools

The "**Calibration Tools**" dialog box (Fig. 3.1) provides useful tools to carlibrate new CaWaQS model. These features are not all operative yet. Indeed the majority of them were not taken tken into account for the refactoring of T1 2026. Anyway they are already designed in the code. A minimal restucturing and wiring would be neededto bring them operative back.

> All the not yet operative functionalities are labelled in the userguide with the acronym WIP (Work in Progress) or with a /!\ symbol in QGIS.

- "**Modify a parametrisation file**" (Sec. 3.1.1) - WIP: tool for modifying the parameterisation files of the various modelled compartments;
- "**Statistical Criteria**" (Sec. 3.1.3): a tool for calculating model calibration performance criteria;
- "**Run-off coefficient calculation**"(Sec. 3.1.4) WIP : a tool for calculating simulated run-off coefficients;
- "**Compare sim/obs**"(Sec. 3.2) WIP: a tool for visualizing simulated and observed time series.

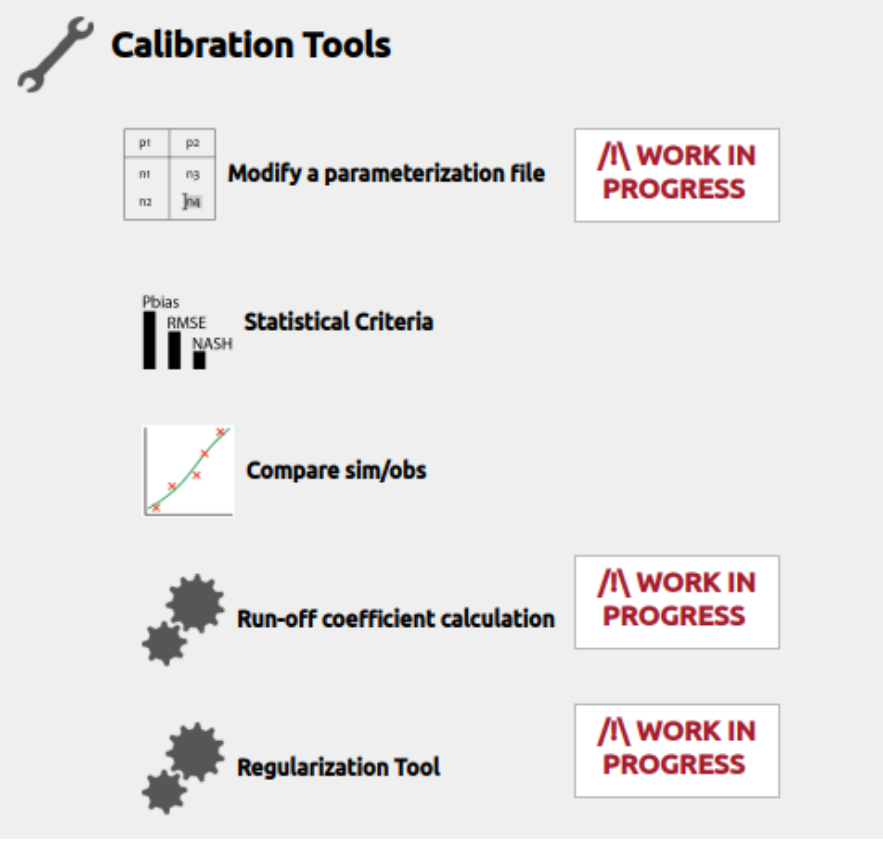
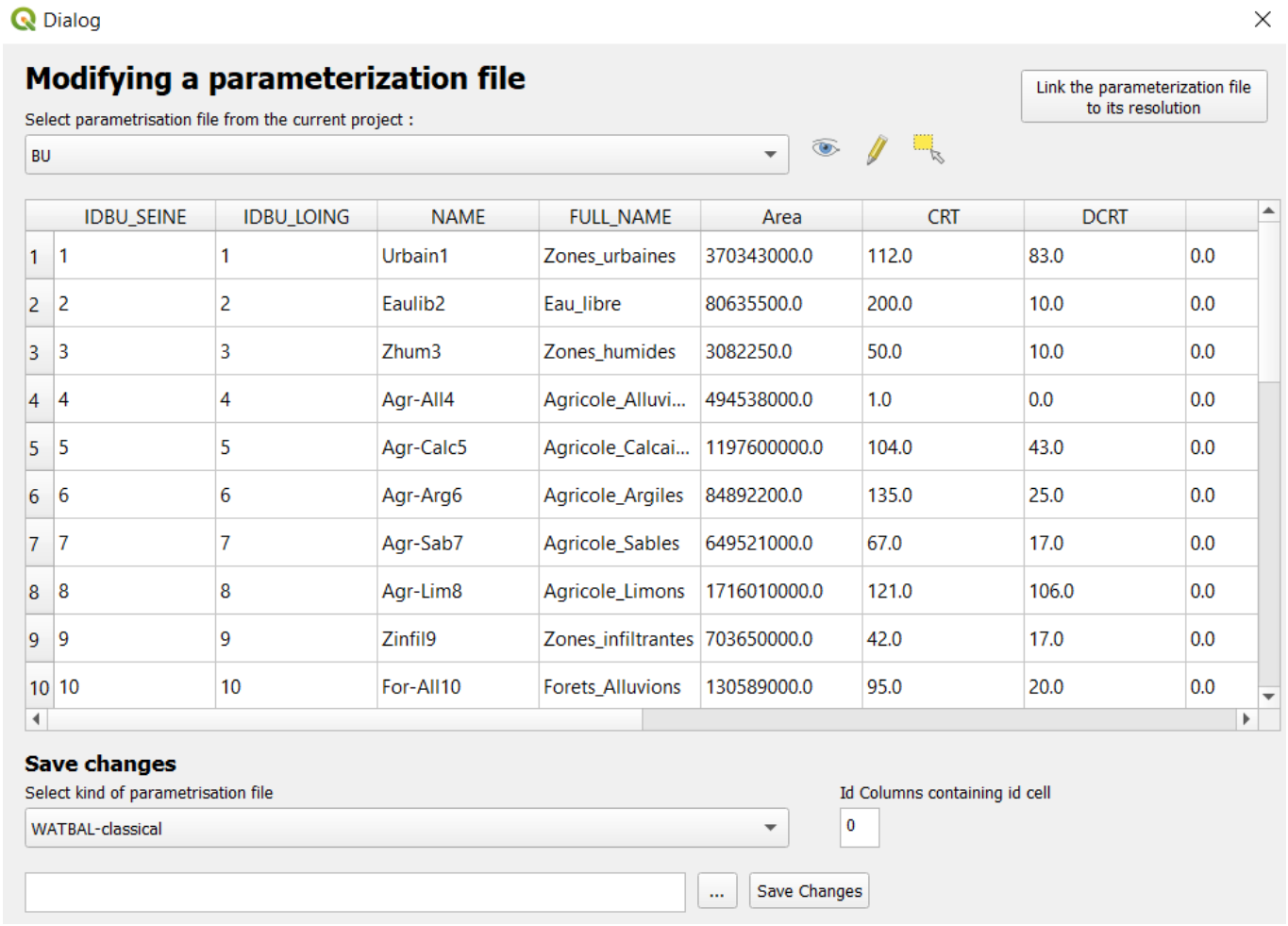


Figure 3.1: Calibration tools dialog box.

3.1.1 WIP: Modifying Parameterisation Files

Parameterisation files can be modified from the "**Modifying a parametrization file**" dialog box (Fig. 3.2). The resolution will depend on the calibration file to be selected from the drop-down menu. The table can be displayed by clicking on .

If the displayed attribute table does not contain the fields of the parameterisation file, the resolution can be linked to its parameterisation file in the "**Link resolution to its parameterisation file**" dialog box, accessible by clicking on Link the parameterization file to its resolution. The use of this dialog box is described in Sec. 3.1.2.



	IDBU_SEINE	IDBU_LOING	NAME	FULL_NAME	Area	CRT	DCRT	
1	1	1	Urbain1	Zones_urbaines	370343000.0	112.0	83.0	0.0
2	2	2	Eaulib2	Eau_libre	80635500.0	200.0	10.0	0.0
3	3	3	Zhum3	Zones_humides	3082250.0	50.0	10.0	0.0
4	4	4	Agr-All4	Agricole_Alluvi...	494538000.0	1.0	0.0	0.0
5	5	5	Agr-Calc5	Agricole_Calcai...	1197600000.0	104.0	43.0	0.0
6	6	6	Agr-Arg6	Agricole_Argiles	84892200.0	135.0	25.0	0.0
7	7	7	Agr-Sab7	Agricole_Sables	649521000.0	67.0	17.0	0.0
8	8	8	Agr-Lim8	Agricole_Limons	1716010000.0	121.0	106.0	0.0
9	9	9	Zinfil9	Zones_infiltrantes	703650000.0	42.0	17.0	0.0
10	10	10	For-All10	Forets_Alluvions	130589000.0	95.0	20.0	0.0

Figure 3.2: Dialog box for modifying parameterisation files.

Once the parameters have been modified in the displayed table, it can be saved. The new file path can be specified by selecting ... . Specify the identifier of the column containing the cell identifier or name[1].

[1]ATTENTION: As a reminder, column identifiers start at **0**.

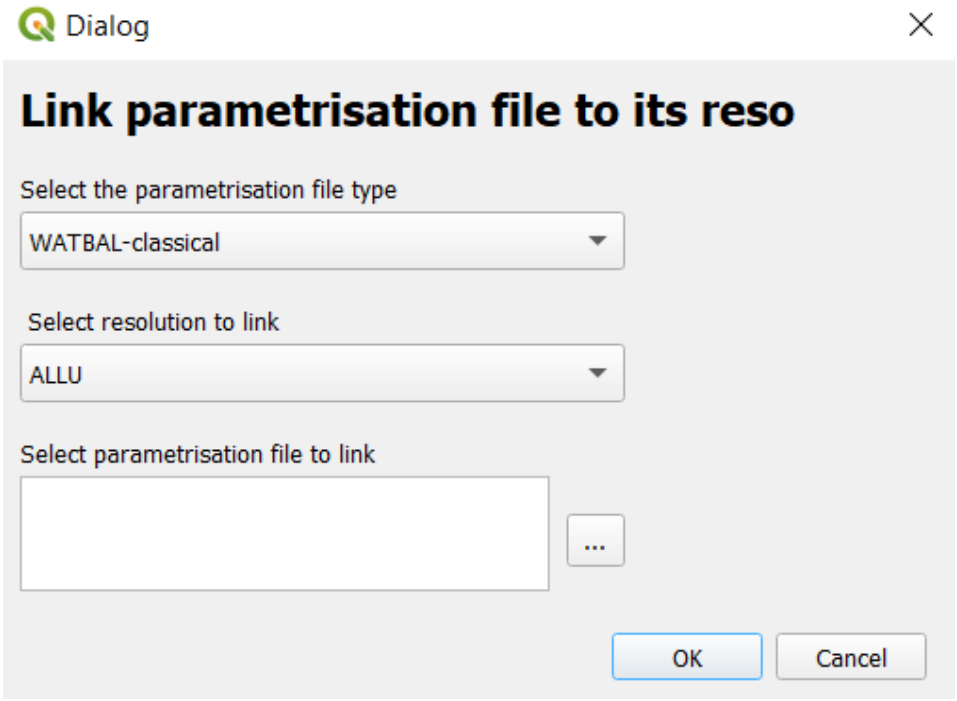


Figure 3.3: Dialog box for linking an attribute table to its resolution.

3.1.2 WIP: Linking an Attribute Table to its Resolution

The parameters from the parameterisation file will be directly imported as fields in the attribute table of the relevant resolution file. To create this link, a parameterisation file type must be chosen (based on the compartment and the type of parameterisation file), the CaWaQS resolution, and the path to this parameterisation file. Different types of parameterisation files exist and are listed in table 3.1.

Compartment	Parameterisation file type	Parameters listed in the table
`WATBAL`	Classical	CRT, DCRT, R, FN, CQR, CQRMAX, CQI, CQIMAX and RNAP
`WATBAL`	Infiltration Excess	CRT, DCRT, R, FN, CQR, CQRMAX, CQI, CQIMAX, RNAP and RINF
`AQ`	Transmissivity	T
`AQ`	Storage (storage coefficient)	S
`AQ`	Conductance	C

Table 3.1: Parameterisation files supported by `CaWaQS-Viz`.

3.1.3 Statistical Criteria: Calculation of Statistical Performance Criteria at Observation Points

The "**Statistical Criteria**" dialog box (Fig. 3.4) returns a GIS layer following the geometries of the observation resolution inherent to the analysed variable. Each field corresponds to a given statistical criterion. The selected statistical criteria are calculated over the period specified in the "**Period for calculating statistical criteria**" section.

3.1.3.1 Statistical criteria calculated by the "Statistical criteria" dialog box

"Pbias" : Bias [%]

$$PBIAS = \frac{\sum(\hat{y}_i - y_i)}{\sum y_i}.100 \qquad \begin{array}{ll} y_i & \text{: simulated variable at time } i \\ O_i & \text{: observed variable at time } i \end{array}$$

"Average SIM/OBS"

$$A = \overline{\hat{y}} - \overline{y} \qquad \begin{array}{ll} \overline{\hat{y}} : & \text{mean of simulated variables} \\ \overline{y} : & \text{mean of observed variables} \end{array}$$

"Nash" : Nash-Sutcliffe Model Efficiency (Nash and Sutcliffe, 1970)

$$NSE = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2} \qquad \begin{array}{ll} y_i : & \text{actual variable at time } i \\ \hat{y}_i : & \text{predicted variable at time } i \end{array}$$

"RMSE" - Root Mean Squared Error

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \quad \text{where} \qquad \begin{array}{ll} n : & \text{Total number of simulation days} \\ y_i : & \text{actual variable at time } i \\ \hat{y}_i : & \text{predicted variable at time } i \end{array}$$

"KGE" - Kling-Gupta Efficiency (Gupta et al., 2012)

$$KGE = 1 - \sqrt{(r-1)^2 + (\alpha - 1)^2 + (\beta - 1)^2} \qquad \begin{array}{ll} r & \text{: Pearson correlation coefficient between the simulated and observed variable series} \\ \alpha & \text{: ratio of simulated and observed standard deviations} \\ \beta & \text{: mean bias between simulated and observed series} \end{array} \tag{3.1}$$

The following criteria are calculated automatically: $\overline{y}$, $\overline{\hat{y}}$, $\frac{\overline{\hat{y}}}{\overline{y}}$, σy, $\sigma \hat{y}$, $\frac{\sigma \hat{y}}{\sigma y}$.

3.1.3.2 Output of the "Statistical Criteria" dialog box

The dialog box generates statistical criteria:

- at observation points,

- at the scale of the aquifer layer in the case of a simulation with an aquifer compartment;
- at the scale of the hydrosystem in the case of a simulation with an aquifer compartment.

Statistical criteria calculated at observation points

The dialog box generates the statistical criteria table at observation points in `shapefile` and `text` format. The Shapefile is introduced directly into the current `QGIS` project, while the text file will be found in the `../PostProcessing/STATS_CRITS` directory. The output criteria are contained in the fields of each measurement point under the names defined in table 3.2.

Statistical criteria calculated at the scale of aquifer units

In the case of a calculation of statistical criteria on the aquifer compartment, a `text` file is saved in the "`Post-Processing`" directory under `../PostProcessing/STATS_CRITS/AQ_crits_xxxx_xxxx.txt`. This file lists the statistical criteria from table 3.2 defined at the scale of each aquifer unit.

Statistical criteria calculated at the hydrosystem scale

Statistical criteria are defined at the hydrosystem scale for the groundwater hydraulic head. These criteria are written in the file `../PostProcessing/STATS_CRITS/AQ_crits_xxxx_xxxx.txt`.



All shapefile outputs generated by CaWaQS-Viz are temporary. If they are not saved, they will no longer be accessible when the project is reopened.

Output table field name	Statistical criterion
"`NObs`"	Number of observations over the selected period
"`AObs`"	Mean of observed variables
"`ASIM`"	Mean of simulated variables
"`SUM_RATIO`"	Ratio of the sum of simulated variables to the sum of observed variables
"`StdObs`"	Standard deviation of observed variables
"`StdSim`"	Standard deviation of simulated variables
"`STD_RATION`"	Ratio of standard deviations of observed and simulated variables
"`PBAIS`"	Bias [%]
"`AVERAGE SIM/OBS`"	Mean ratio between simulated and observed variables
"`KGE`"	Kling-Gupta Efficiency
"`NASH`"	Nash-Sutcliffe Model Efficiency
"`RMSE`"	Root Mean Squared Error

Table 3.2: Field names of output tables for each statistical criterion

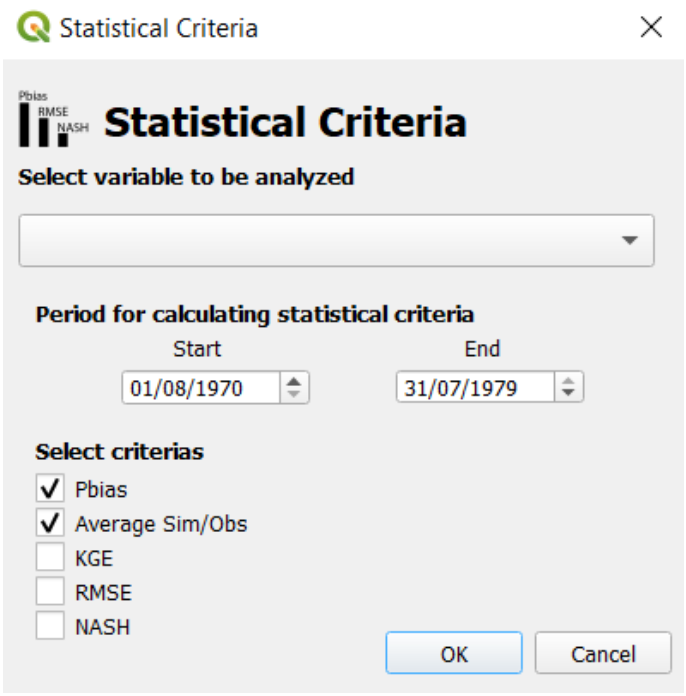


Figure 3.4: "Statistical Criteria" dialog box.

3.1.4 WIP Run-off coefficient calculation: Calculation of the Run-off Coefficient at Flow Measurement Stations

This dialog box calculates the simulated and observed run-off coefficient at flow measurement stations.

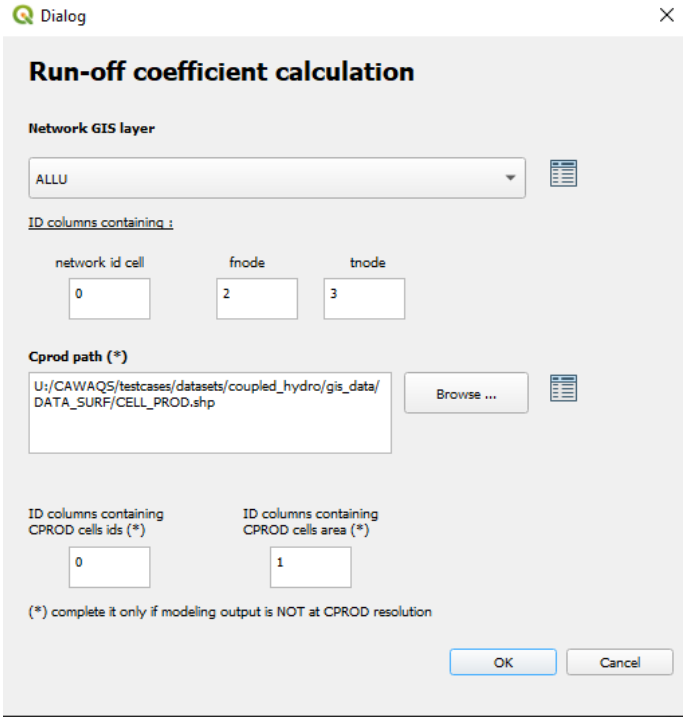


Figure 3.5: Dialog box for calculating the run-off coefficient.

This dialog takes as input the resolution of the surface hydrological scheme at the river reach scale and the CaWaQS surface resolution of the production cells (unit catchments or `Cprod`). For the former, the column number containing the reach identifiers, `Fnode` and `Tnode` must be specified. For the latter, the identifiers of the fields containing the production cell identifier, and the cell area.

If the surface resolution defined in the CaWaQS-Viz configuration is already that of production cells, it will not be necessary to fill in the lower part of the dialog box.

Two new resolutions (*shapefiles*) will be returned by the dialog box and added to the current GIS project:

- "`CATCH`" which contains the catchments drained by each measurement station;
- "`CATCH_SURF`", the product of the intersection between the surface resolution defined in the CaWaQS-Viz configuration and "`CATCH`". Its attribute table contains several fields:
  - "`CATCH_ID`" and "`CATCH_SURF`": the identifier of the catchment drained by the station and its area;
  - "`ID_OBS`": the identifier of the measurement station;
  - "`SURF_ID`": the area of the surface cell from the surface resolution defined in the CaWaQS-Viz configuration;
  - "`SURF_SURF`": its area;
  - "`INTER_ID`": the identifier of the intersection cell;
  - "`INTER_SURF`": the intersection area.

These resolutions are saved in temporary format. It is left to the user to save them for future reuse. If CaWaQS-Viz recognises these resolutions in the current QGIS project, they will be reused for calculating run-off coefficients.

These coefficients can be found in the attribute table of the flow observation layer specified at the configuration step. Two new fields are then added: "QruissSim" and "QruissObs" designating respectively the run-off coefficient simulated by CaWaQS and that observed at the measurement stations.

3.2 Compare sim/obs: Comparison of Observed and Simulated Time Series

This calculation module enables comparison between simulations and observations through several types of output:

- by clicking on "PDF printout", a pdf file is generated. The simulated and observed time series are presented with the statistical criteria listed in table 3.2;
- by clicking on "Interactiv graphics", an html format reproduces the time series presentation format of the pdf interactively in the default browser;
- by clicking on "Statistical criteria", the "Statistical Criteria" dialog box (3.1.3) opens. The parameters specified in the "Compare Sim-Obs" dialog box (dates, compartment and observation unit of measurement) are transferred to the "Statistical Criteria" dialog box.

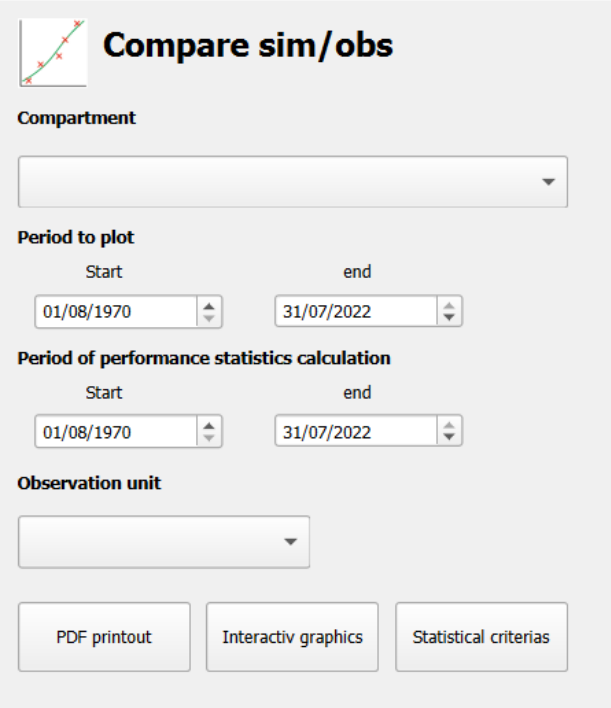


Figure 3.6: "Compare sim/obs" dialog window.

The compartment type (AQ = aquifer and HYD = hydrographic network) is to be selected from the drop-down list located in the upper part of the dialog box (Fig. 3.6). The time series printing periods and those for calculating statistical criteria must also be specified. The unit of flow observations, used for processing simulations of the "HYD" compartment, must be specified in the "Observation unit" drop-down menu. It is

optional for processing the "`AQ`" compartment.

An information message will be displayed in the QGIS interface at the end of this task and will point to the post-processing directory. The `.pdf` files resulting from analysis of the `HYD` compartment will be named "`SIM_OBS_HYD.pdf`" by default. Those from aquifer compartment analysis will be named "`SIM_OBS_AQ.pdf`". Each page of this document corresponds to a measurement station.

3.3 `Spatial Map`: Spatial Mapping of Hydrological Variables

This tool spatialises the surface water balance in `.shp` format. The balance can be produced at an annual, monthly or daily frequency (to be specified in the `frequence` field). Temporal aggregation can be performed by mean, sum, maximum or minimum (to be specified in the `agregator` field).

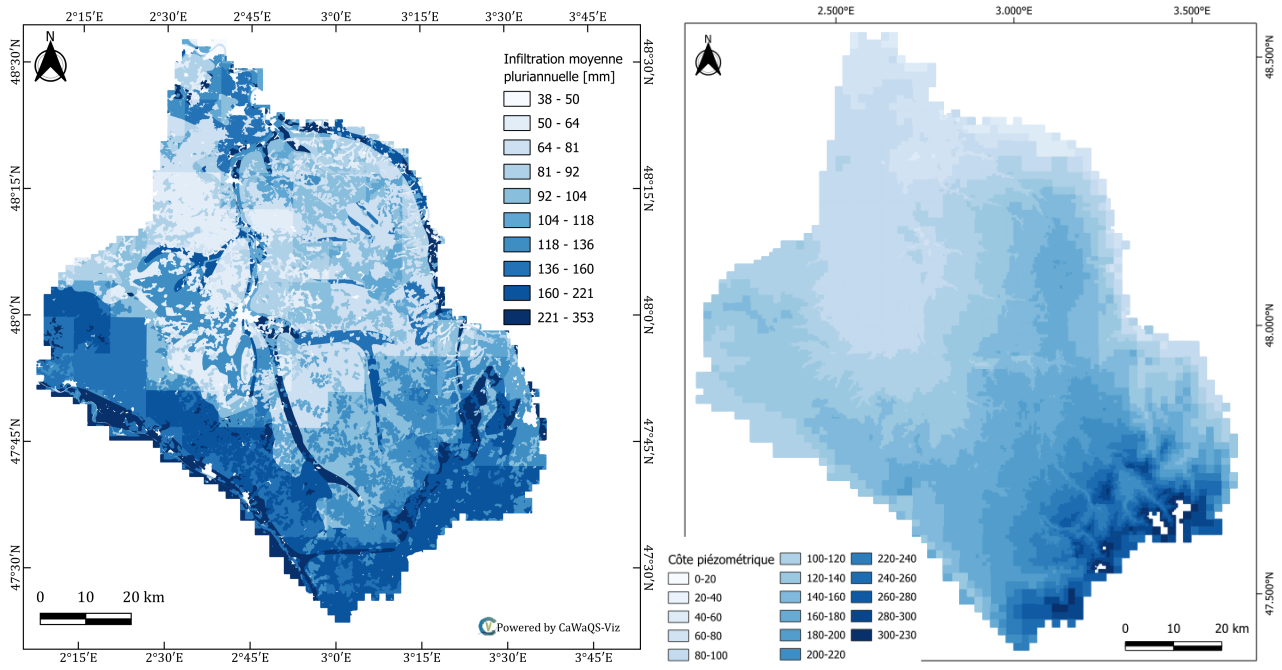


Figure 3.7: Spatialisation of mean inter-annual infiltration flux [mm] (a) and piezometry [$mNGF$] (b) simulated at the scale of the Loing hydrosystem

The values taken by the hydrological variable to be spatialised are contained in the attribute table of the output `.shp` resolution. The table has as columns the year identifiers for annual aggregation, the month identifiers for monthly aggregation and finally the date for daily time step output. A multi-year mean aggregation is also possible by checking the `Pluriannual` box, thus establishing a multi-year or multi-year monthly balance.

- `WATBAL`: the precipitation, AET, runoff and infiltration variables (3.7a);
- `AQ`: hydraulic heads (3.7b), viewable either on a given aquifer mesh layer, or on the outcropping aquifer

cells (unique mesh, created by CaWaQS-Viz).

Variables from the `WATBAL` compartment are expressed in $mm\,j^{-1}$ or $mm\,an^{-1}$ depending on the chosen aggregation mode. Variables from the `AQ` compartment are expressed in metres.

3.4 `Budget Bar Plot`: Hydrological Budget Visualization

Simulated hydrological budgets can be visualized from this dialog box. They are constructed in accordance with the specified period. The `Frequences` field specifies the water balance calculation time step (multi-year, annual, monthly or daily), while `agregator` specifies the temporal aggregator (mean, sum, maximum, minimum). The water balance is spatially aggregated by a mean of hydrological variables over all surface cells.

Upon execution of this dialog box, an `HTML` page opens in your browser and displays an interactive hydrological budget chart. A static image (cf. Fig. 3.8) of this same chart is saved in the post-processing directory in a sub-folder `../BUDGET` of the post-processing directory.

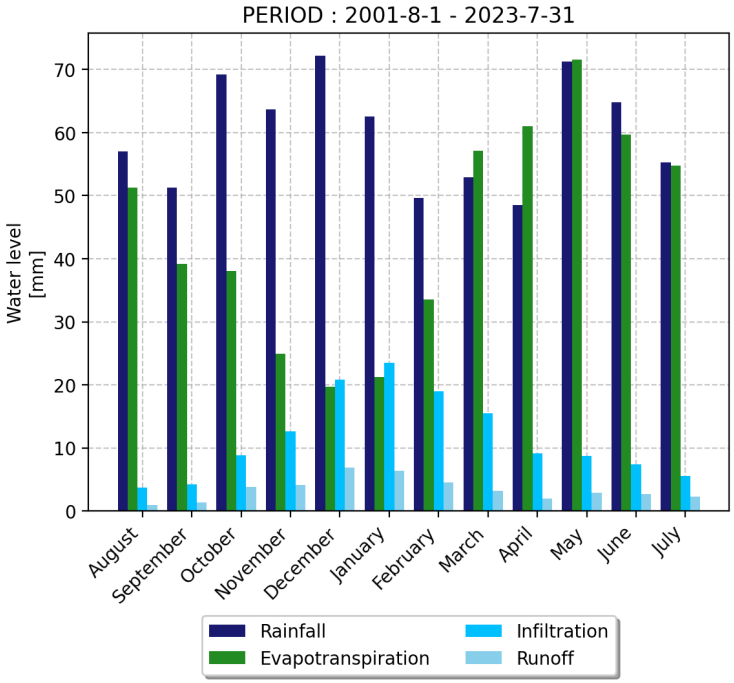


Figure 3.8: Example of a graphical output for the simulated multi-year monthly hydrological budget

3.5 `Hydological Regime`: Hydrological Regime Visualization

The simulated hydrological regimes of the hydrosystem are constructed from the "`Hydrological Regime`" dialog box (cf. Figure 3.9). Hydrological regimes are calculated over the period specified in the "`Start`" and "`end`" input fields. The variable to be analysed is to be specified in the "`Hydrological variable to plot`" drop-down menu. The output type is chosen by the user by checking the "`static plots (PNG)`" box for graphical outputs as individual images for each measurement point, "`static plots (PDF)`" for a PDF output compiling the hydrological regimes of all measurement points, and finally "`interractives plots`" for an output in the default browser in the form of interactive charts (3.10). A dialog box alerts the user when the calculation module has finished executing.

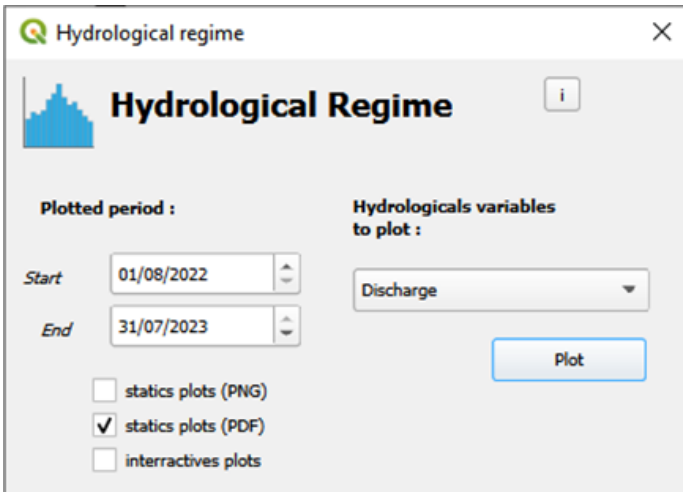


Figure 3.9: "`Hydrological Regime`" dialog box for calculating multi-year monthly hydrological regimes.

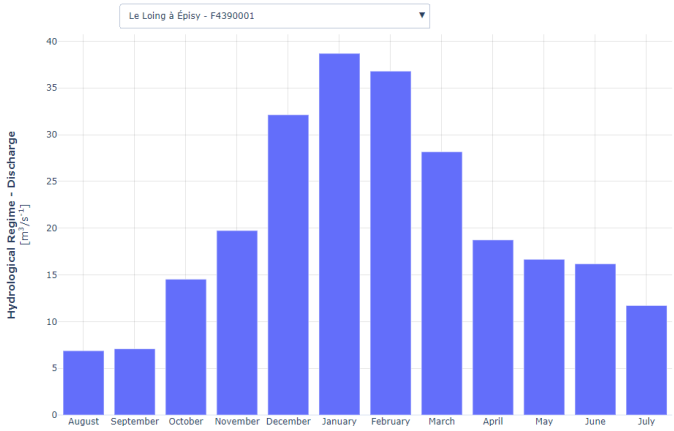


Figure 3.10: Example of a graphical output for the simulated multi-year monthly hydrological regime

3.6 Extract Data from a sub area: Extraction of Results in a Polygon

The "**Extract Data from a Sub Area**" dialog box (Fig. 3.11) allows extracting simulation results over a defined zone. This zone can be specified either as a polygon from a `QgsVectorLayer` loaded in the project, or as a shapefile imported from a directory.

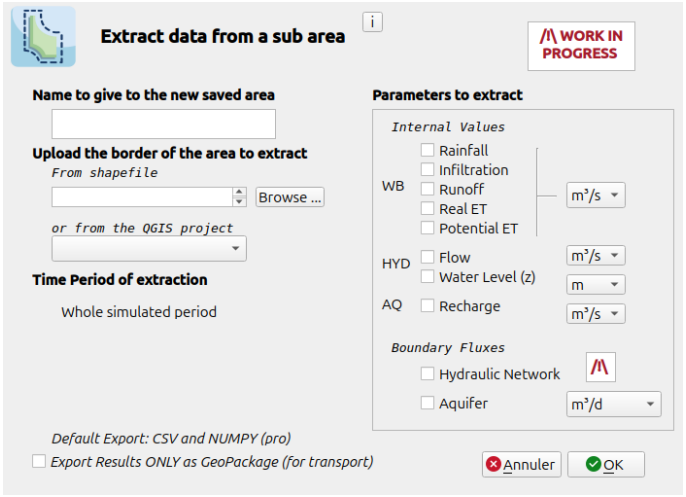


Figure 3.11: "**Extract Data from a Sub Area**" dialog box.

Two families of variables can be extracted, each driven by its own set of tickboxes and each producing its own output bundle: the *internal values* (Sec. 3.6.4), i.e. the simulated variables of the **WATBAL**, **HYD** and **AQ** compartments inside the polygon, and the *boundary values* (Sec. 3.6.5), i.e. the fluxes crossing the polygon boundary. Both families belong from an independent "**GeoPackage**" tickbox that switches all the output format.

3.6.1 Parameters

From the dialog box, the following parameters can be specified for extraction:

- Name of the new extraction file (also used as the name of the folder created in `PostProcessing/DATA_EXTRACTED/`)
- Extraction polygon: either a shapefile or a `QgsVectorLayer` present in the project;
- Parameters to extract (tickboxes);
- The unit of measure, chosen from the combo box next to each parameter (see Sec. 3.6.3);
- Whether the output format should be a GeoPackage or the default loose-file layout (Sec. 3.6.2).

3.6.2 Outputs

For each extraction, result files and `QgsVectorLayers` are produced. Files are saved in the output directory defined in the main analysis parameters, under `PostProcessing/DATA_EXTRACTED/<Extraction_Name>`, which contains the following sub-folders:

- **CONFIG/**: configuration files for rerunning simulations WIP;
- **OUTPUTS/**: CSV or GeoPackage files containing the results;

- **TEMP/**: NumPy files used to rerun analyses.

The "**GeoPackage**" tickbox does not *add* a file, it *replaces* the loose files. The two modes are mutually exclusive:

- **Unticked (default)**: one CSV per checked parameter in `OUTPUTS/` and one `.npy` binary per parameter in `TEMP/` (plus a polygon-total CSV per parameter when weighting is enabled). No GeoPackage is written.
- **Ticked**: a single GeoPackage in `OUTPUTS/` and nothing else — no CSV, no `.npy`.

3.6.3 Units

Each parameter carries its own unit, picked from the combo box next to its checkbox. The **WATBAL** fluxes, the **AQ** "**Recharge**" and the **HYD** "**Flow**" are volumetric (m^3/s or m^3/d); the **HYD** "**Water Height**" is a length (`m` or `cm`) and therefore skips the per-day volumetric conversion. The **AQ** subsurface (boundary) fluxes have a separate unit combo of their own, described in Sec. 3.6.5.

3.6.4 Parameters — Internal Values

The following variables can be extracted for the surface (**WATBAL** compartment of CaWaQS):

- "**Rainfall**";
- "**Real EvapoTranspiration (ET)**";
- "**Infiltration**";
- "**Runoff**";
- "**Potential EvapoTranspiration (ET)**".

For the river system (**HYD**):

- "**Flow**";
- "**Water Height**";

For the aquifer (**AQ**):

- "**Recharge**".

3.6.4.1 The internal-values GeoPackage

When the "**GeoPackage**" box is ticked, all the checked compartments are bundled into one single file, `<area>_InternalValues_<years>.gpkg`, written in `OUTPUTS/`. It contains:

- one `cells_<compartment>` vector layer per mesh (e.g. `cells_WATBAL`, `cells_AQ`, `cells_HYD`) holding the masked geometry — **HYD** reaches are clipped to the polygon boundary;
- a `daily_values` table with one row per (compartment, cell, date, parameter) tuple; it is joined to a mesh layer on the pair (`compartment`, `cell_id`);
- a `provenance` table with one row per compartment;
- a `polygon_total_<compartment>_<param>` table per parameter, when weighting is enabled.

3.6.5 Parameters — Boundary Values

- "**Aquifer**" subsurface (horizontal) flows: the fluxes crossing the vertical faces of the aquifer cells that lie on the polygon boundary;
- "**Hydraulic network**" WIP: flow of the reaches crossing the boundaries of the mask.

3.6.5.1 Unit and sign convention

The "**Subsurface flows**" checkbox has its own unit combo box, offering two options that differ in *nature*, not merely in scale:

- m^3/d (default): a daily flow **rate** (the raw m^3/s multiplied by 86 400), giving one value per day;
- $m^3/month$: the actual total **volume** (m^3) that crossed each boundary during each calendar month. The time axis is re-binned to one row per month, each value being the sum, over that month's days, of the daily $m^3/s \times 86\,400$. Real month lengths (28, 29, 30 or 31 days) are honoured.



In $m^3/month$ mode, a partial first or last month totals **only its simulated days**. A first or last month whose volume looks small compared to its neighbours is therefore correct, not an error.

The sign convention travels with the data — in the CSV header line and in the GeoPackage `provenance` table: **positive means a flux entering the cell**.

3.6.5.2 Default mode: the face-flux CSV

With the "**GeoPackage**" box unticked, the boundary fluxes are written to a loose face-flux CSV. Faces are named according to the convention `nCell(ID_ABS)_direction_unit`, for example `3323_west_m3d`. This is the **only** mode that exposes the per-face *gross* detail, i.e. each face's own value taken separately.

3.6.5.3 GeoPackage mode: the AQ-boundary GeoPackage

The AQ-boundary GeoPackage is exclusive in the same way as the internal-values one: when the box is ticked, a single `<area>_AqBoundary_<years>.gpkg` is written *instead of* the loose face-flux CSV. It carries:

- one `cells_AQ_layer<id>` geometry layer per aquifer layer;
- a values table — `daily_values` in m^3/d mode, `monthly_values` in $m^3/month$ total-volume mode.

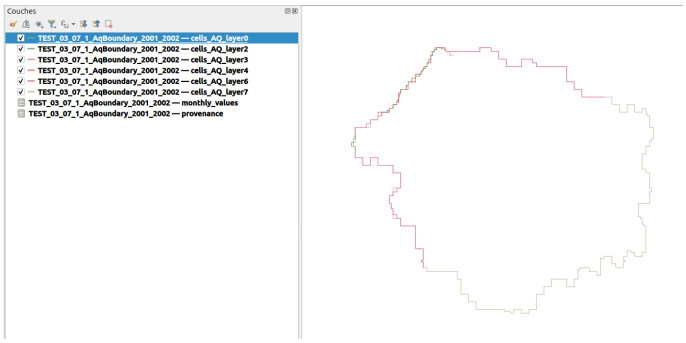


Figure 3.12: "`AQ boundary geopackage`".

Both the geometry layers and the values table carry a `faces` column: the comma-separated list of the cardinal directions each boundary cell touches, for example `'north,west'`. Each row remains **one net series per cell**; the `faces` column simply says which directions contributed to it. The `faces` column also appears in the attribute table of the QGIS borders layer registered in the project, regardless of the GeoPackage mode.

Per-face structure columns. Inside the GeoPackage file only, seven additional columns spread that summary across queryable fields. They appear on both the `cells_AQ_layer<id>` geometry layers and the values table, but **not** on the registered QGIS borders layer, which keeps only `faces`.

- `n_faces`: how many cardinal sides the cell borders (1 to 3);
- `face1_orient/face1_outid ... face3_orient/face3_outid`: two columns per face slot.

`faceN_orient` is the N-th bordered direction; slots beyond `n_faces` are left blank. `faceN_outid` is filled **only** when that face's flux was sourced from the outside cell — the *coarse-inside* case, where the reported value came from the smaller outside neighbours, whose identifiers are then comma-joined into the field. It stays blank for a normal own-face read.

3.6.6 Using the extraction from a notebook

The two families of the dialog are also available from a notebook (chapter 4) through the client API:

- `client.mask_internal_values(...)` for the internal values;
- `client.mask_aq_boundary(...)` for the aquifer boundary fluxes.

The dialog tickboxes are then replaced by explicit arguments.

3.6.6.1 Internal values: the `specs` tuples

`mask_internal_values` takes a `specs` list in which every entry is a 4-tuple `(compartment, outtype, param, unit)`:

- `compartment`: `"WATBAL"`, `"HYD"` or `"AQ"`. The compartment must be loaded by the client (`IDS_COMPARTMENTS`: 1 = `AQ`, 2 = `HYD`, 3 = `WATBAL`);
- `outtype`: the CaWaQS binary the parameter is read from (`MB`, `Q` or `H`);
- `param`: a record name of that binary;
- `unit`: the output unit of *that* spec. Each tuple carries its own unit; there is no global unit argument.

Tab. 3.3 lists every accepted combination.

compartment	outtype	param	unit
WATBAL	MB	rain, etp, runoff, inf, etr, direct_sout, stocksoil, stockinf, stockruiss, error	m3/s, m3/j
HYD	Q	discharge	m3/s, m3/j
HYD	H	water_height, water_level	m, cm
AQ	H	piezhead	m, cm
AQ	MB	h_end, dv_dt, flux_x_one, flux_x_two, flux_y_one, flux_y_two, flux_z_one, flux_z_two, recharge, uptake, flux_direchlet, flux_neumann, surf_overflow, flux_riv_to_aq, err, err_rel	m3/s, m3/j

Table 3.3: Accepted (compartment, outtype, param, unit) tuples for **mask_internal_values**.

The dialog tickboxes of Sec. 3.6.4 map to the tuples as follows: "**Rainfall**" = rain, "**Real EvapoTranspiration (ET)**" = etr, "**Potential EvapoTranspiration (ET)**" = etp, "**Infiltration**" = inf, "**Runoff**" = runoff, "**Flow**" = discharge, "**Water Height**" = water_height, "**Recharge**" = ("AQ", "MB", "recharge", ...). The other parameters of the table are available from the notebook only.

```
MASK_SPECS = [
    ("WATBAL", "MB", "runoff",         "m3/j"),
    ("HYD",    "Q",  "discharge",      "m3/s"),
    ("HYD",    "H",  "water_level",    "m"),
    ("AQ",     "MB", "recharge",       "m3/j"),
    ("AQ",     "MB", "surf_overflow",  "m3/j"),
]
```

Code 3.1: Example of specs for **mask_internal_values**.



A unit must belong to the right family. water_height, water_level and piezhead only accept m or cm; every other parameter only accepts m3/s or m3/j. A wrong pairing raises a TypeError. The tokens m3/mois, m3, mm/j and l/s are **not** accepted for internal values.

A few behaviours worth knowing:

- **AQ** specs are resolved on the *outcropping* (top) aquifer layer, not per layer. This is the right support for recharge and surf_overflow. For the lateral fluxes across the polygon boundary, use **mask_aq_boundary** (below) instead of flux_x_*/flux_y_*, which would return every cell *inside* the polygon;
- weighted=True is ignored for length units (m, cm) and for all **HYD** specs: reaches always contribute their raw value;
- h_end and err_rel are accepted with a volumetric unit, but are not physical fluxes: interpret them with care.

3.6.6.2 Boundary values: `mask_aq_boundary`

`mask_aq_boundary` takes **no** `specs`: it always reads the four lateral face fluxes of the `AQ_MB` binary (`flux_x_one` = west, `flux_x_two` = east, `flux_y_one` = south, `flux_y_two` = north) on the cells lying on the polygon boundary, split per aquifer layer. The only unit argument is `unit`:

- `"m3/j"` (default): daily rate in m^3/d — the m^3/d option of the dialog;
- `"m3"`: calendar-month total volume, written to a `monthly_values` table — the $m^3/month$ option of the dialog;
- `"m3/mois"` (notebook only): an *average-month* rate, i.e. every daily value multiplied by $86\,400 \times 365.25/12$, keeping one row per day.

```
result = client.mask_aq_boundary(
    polygon=polygon, polygon_crs=polygon_crs,
    syear=START_YEAR, eyear=END_YEAR,
    output_dir=CWV_output_directory, temp_dir=TEMP_DIR,
    area_name="MY_AREA_AQ_BOUNDARY",
    write_geopackage=True,
    unit="m3",          # "m3/j" (default), "m3" or "m3/mois"
)
```

Code 3.2: Example call of `mask_aq_boundary`.



Both operations need the `AQ` meshes: every aquifer layer referenced by the geometry configuration must be listed in `layer_paths`, and `1` must be included in `IDS_COMPARTMENTS`. Otherwise the notebook cell prints `Skipped`.

CHAPTER 4

Using CaWaQS-Viz from a Notebook (without QGIS)

Contents

4.1 Using CaWaQS-Viz from a Notebook (without QGIS) - Prerequisites 34
4.2 Setting up the environment 34
4.3 Running on a remote server 34
4.4 Choosing a template 35
4.5 Workflow 35

4.1 Using CaWaQS-Viz from a Notebook (without QGIS) - Prerequisites

The scientific backend of CaWaQS-Viz, `HydrologicalTwinAlphaSeries` (HTAS), can be driven directly from a Python notebook, **without QGIS**. This exercises the same public API that the QGIS dialogs use, on a real CaWaQS simulation output. Beyond convenience, these notebooks are the guarantee that the backend stays QGIS-independent: anything that runs from a plain Python notebook today will also run on the future CaWaQS-Viz server (chapter 7). The notebooks live under `interface/notebooks/`.

- a local checkout of `cawaqsviz` with the `HydrologicalTwinAlphaSeries` submodule populated (Sec. 1.1);
- a readable CaWaQS simulation output directory: the binaries under the CaWaQS output directory, the `ConfigGeometry .json` shipped with the simulation, and the matching `shapefiles/geopackages`.

4.2 Setting up the environment

The recommended way is a dedicated `pixi` environment that pulls `jupyterlab` and an editable install of HTAS, but *not* QGIS. From the repository root:

```
pixi install -e notebook          # create the notebook environment (once)
pixi run -e notebook notebook     # launch Jupyter Lab, rooted at interface/notebooks/
```

Jupyter Lab opens in your browser; double-click a notebook and run.

`pixi` is only a convenience. HTAS is a standard pure-Python package (no QGIS, no conda-only dependencies), so any virtual environment with `jupyterlab` works:

```
# venv + pip
python -m venv .venv && source .venv/bin/activate
pip install jupyterlab
pip install -e external/HydrologicalTwinAlphaSeries
jupyter lab interface/notebooks/
```

The editable install of `external/HydrologicalTwinAlphaSeries` pulls in HTAS's runtime dependencies (`geopandas`, `numpy`, `pandas`, `matplotlib`, `plotly`, `scipy`, `shapely`). The only thing `pixi` adds is a fully pinned, reproducible environment.

4.3 Running on a remote server

When the simulation lives on a compute server, you want the notebook to *run* there (next to the data) but *edit* it on your laptop. The recommended way is the VS Code **Remote-SSH** extension, which starts the kernel on the server and opens the `.ipynb` as a normal editor tab — no browser, no token, no tunnel. Register the environment once, on the server, from the repository root:

```
pixi run -e notebook register-kernel
```

This makes a kernel called "CaWaQS (pixi notebook)" appear in the VS Code kernel picker (select Kernel>jupyter Kernel> CaWaQS (pixi notebook)). You then connect with "`Remote-SSH: Connect to Host`", open the `.ipynb`, select that kernel, and run the cells.



If VS Code is not available, an SSH tunnel is a working fallback: start `jupyter lab --no-browser` on the server, forward the port with `ssh -L 8888:localhost:8888`, and open the printed URL locally. See `interface/notebooks/README.md` for the full walk-through.

4.4 Choosing a template

The `interface/notebooks/` folder ships **two templates**. Both share the same parameters cell and the same `layer_paths` contract — they differ only in *which* HTAS surface they call:

- `template_client.ipynb` — the coarse, dialog-shaped API (`HydrologicalTwinClient`). Copy this to **reproduce what a QGIS dialog does** (e.g. `budget_barplot`, `spatial_map_aq`, `mask_hyd_boundary`). This is also the surface a future HTTP server will expose;
- `template_developer.ipynb` — the low-level verb API (`HydrologicalTwin`, verbs `fetch` / `mask` / `transform` / `render`). Copy this to **build your own pipeline**, inspect intermediate arrays, or call a primitive not yet available in the client.

Rule of thumb: *"a plot that matches a QGIS dialog"* → **client**; *"my own chain of operations"* → **developer**. The two surfaces are the two *entry gates* of HTAS, described in the developer chapter (Sec. 7.4).

4.5 Workflow

1. **Copy the template** — never edit a template in place. Right-click it → "`Duplicate`" and rename the copy (e.g. `seine_2023.ipynb`).
2. **Edit the parameters cell only.** The rest of the notebook is designed to run unchanged.
3. **Clear outputs before committing.** Outputs bloat diffs and can leak paths from your machine. From the repository root:

```
jupyter nbconvert --clear-output --inplace interface/notebooks/<your_notebook>.ipynb
```

CHAPTER 5

Manual Project Configuration (Advanced)

Contents

5.1 Contents of the QGIS Project Used for Processing CaWaQS Simulation Results 37
5.2 Configuring the Project Geometry from an Existing QGIS Project 39
5.3 Configuring Resolutions from a `.json` Configuration File 42



Project setup is now **automated**: on a normal first run the plug-in detects the CaWaQS outputs and fills the geometry configuration for you. The manual path described in this chapter is a **fallback** for advanced users — when the automatic detection cannot resolve the project, or when a configuration has to be built or adjusted by hand. New users do not need it to get started.

The lower part of the "`Settings`" dialog box allows manual configuration of the plug-in. The following elements must be defined:

1. a project name;
2. the modelled compartments;
3. a GIS layer mesh configuration;
4. the modelling output and observation directories[1] if observation points are defined in the project.

The project name will be used to name a sub-folder created in the post-processing directory. The graphical outputs of the CaWaQS-Viz application will be placed in this post-processing folder.

The geometry configuration can also be imported from a `.json` configuration file (by clicking on "`Import a geometry configuration from a .json`") or defined manually *via* the option "`Select resolutions from the current project`".



Note that any manual project and geometry configuration will be exported in `.json` format to the post-processing directory upon execution of the "`Settings`" dialog box.

5.1 Contents of the QGIS Project Used for Processing CaWaQS Simulation Results

The `shapefiles` of the CaWaQS model meshes

The QGIS project used for processing simulations with the CaWaQS-Viz utility must contain:

- the meshes (resolutions) of the compartments in "`shapefile`" format of the model (refer to the CaWaQS documentation[2] to identify the output resolutions of the "`WATBAL`" compartment). The CaWaQS resolution types to import into the GIS project for processing modelling outputs depend on the compartments actually being modelled;
- the observation points of the compartments to be defined in the CaWaQS-Viz extension.

[1] The flow and piezometry observation directories are assumed to be in the same parent directory.

[2] CaWaQS geometry terminology. For more information on the different types of meshes, refer to the model documentation (*i.e.* user guide and technical guide), available on the GitLab repository https://gitlab.com/cawaqs/cawaqs.

	Surface compartment (WATBAL)	Hydraulic compartment (HYD)	Unsaturated compartment (NSAT)	Aquifer compartment (AQ)
CaWaQS resolution	Unit surface cell (`ele_bu`)[3] or Production cell (`Cprod`)	River element (`ele_musk`)	Unsaturated zone grid	Meshes of all aquifer layers
Required fields in the attribute table	Cell identifiers	GIS identifiers of river elements	Cell identifiers	Absolute cell identifiers

Table 5.1: CaWaQS resolutions that can be imported as compartment resolution.



The names of the GIS layers in the project corresponding to aquifers must be identical to those defined in the CaWaQS configuration to ensure correct application behaviour.

The `shapefiles` of extraction points

Extraction of simulated flow and hydraulic head can be performed at extraction points. Two types of extraction points can be defined:

- simple extraction points, which do not include a measurement time series;
- observation points, which are extraction points with measurement data.

Extraction and observation points are contained in two separate `shapefile` files whose attribute tables contain the fields listed in table 5.2.

POINT TYPE	ATTRIBUTE TABLE CONTENTS	OUTPUTS
Extraction point	• Point name • Identifier of the attached cell[4], aquifer layer identifier (if extraction in aquifer)	• Simulated hydrographs and piezometric curves at the extraction point
Observation point	• Point name • Identifier of the attached cell [4]	• Simulated and observed hydrographs and piezometric curves • Objective criteria comparing observations with simulations

Table 5.2: Structure of GIS layers for extraction points and associated graphical outputs

Observation **shapefiles** must be imported into the project and specified in the CaWaQS-Viz configuration if the HYD and/or AQ compartments are being processed. Without observations, the extension will not be functional.

5.2 Configuring the Project Geometry from an Existing QGIS Project

The geometric configuration of the CaWaQS-Viz application can be defined from an existing QGIS project via the "**Application configuration from an existing project**" dialog box in the application settings dialog.

The window is organised into two sections:

- on the left, the CaWaQS compartments are listed (WATBAL, HYD, AQ, NSAT),
- on the right, the parameters to be defined are listed for producing graphical outputs (flow and piezometry) at observation points (extraction points with observations) or extraction points (without observations).

If the application configuration fields are left empty, the compartment concerned is not included in the geometric configuration.

The compartment fields are configured as follows:

The surface compartment (""WATBAL COMPARTMENT"") :

- "**Surface resolution**": output resolution of the surface water balance (CPROD or elebu);
- "**cell id**": the column number in the attribute table corresponding to the cell identifier;
- "**cell area**": the column number in the attribute table corresponding to the cell area (optional parameter).

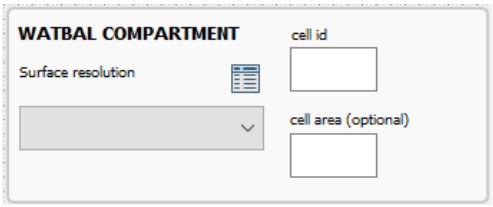


Figure 5.1: WATBAL compartment configuration fields

The hydraulic compartment ("HYD COMPARTMENT")

- "**Hyd resolution**": name of the output resolution of the "**HYD**" compartment (river elements);
- "**GIS Id**": the column number in the attribute table corresponding to the GIS identifier of the cells



Figure 5.2: HYD compartment configuration fields

The unsaturated compartment ("NSAT COMPARTMENT"):

- "`Unsatured zone resolution`": name of the output resolution of the "`NSAT`" compartment;
- "`cell id`": the column number in the attribute table corresponding to the cell identifier.

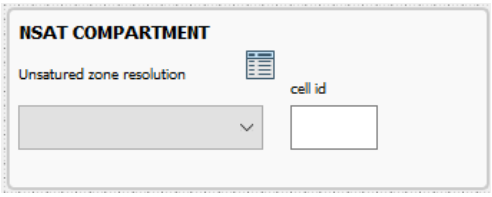


Figure 5.3: NSAT compartment configuration fields

The saturated compartment ("`AQ COMPARTMENT`")

- "`Aquiferes resolutions`": the GIS layers constituting the underground mesh. The selected layer is added by clicking on "`Add`". Layers must be ordered from the uppermost to the lowermost layer using the "`up`" and "`down`" buttons. Finally, a layer can be removed from the list by clicking on "`remove`";
- "`cell id abs`": identifier of the attribute table column containing the absolute cell identifier. This identifier can be homogeneous across GIS layers: in that case, "`homogeneous ID ABS`" must be selected. Otherwise, click on "`heterogeneous ID ABS`" to open dialog boxes allowing you to specify, for each layer, the column containing the ABS ID of the cells (cf. Fig 5.4).

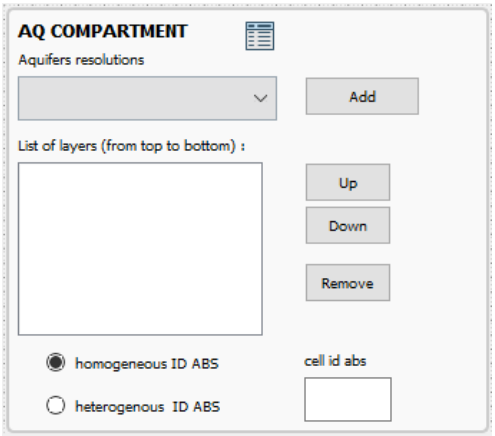


Figure 5.4: AQ compartment configuration fields

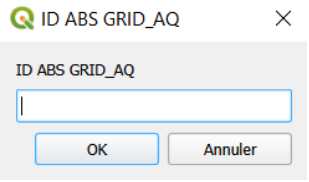


Figure 5.5: Window for selecting the field identifier containing the "`ID ABS`" in the attribute table in the case of heterogeneous field identifiers across layers

Some points of caution:

- **The identifier of the first column of an attribute table starts at 0**. The icon allows viewing the identifiers of the fields listed in the attribute table of the resolution selected in the drop-down list;
- the code (e.g., BSS code) of an observation point must be **unique** among all observation points;
- The geometric configuration of the compartments checked in the project parameterisation dialog box must be defined in the geometry configuration dialog box.

Extraction points - "`Hydrological and Hydrological plots`"

Extraction points can be of two types:

- simple extraction points, which allow viewing simulated variables at those points;
- observation points, which allow comparing simulated variables with observation data at those points.

The configuration of extraction points requires specifying in the "`GIS Layer`" fields the name of the layer in the QGIS project containing the observation points and extraction points according to the relevant `CaWaQS` compartment. The field identifiers to be specified are as follows:

Extraction points of the "`HYD`" compartment

Extraction points

- "`name`": name of the observation point;
- "`ID GIS`": GIS identifier of the observation point to which the observation point must be attached (optional field)

Observation points

- "`code`": identification code of the observation point, must be unique for each point. The observation file for the point must have the same name with the `.dat` extension;
- the "`name`" and "`ID GIS`" fields are also specified for the configuration of observation points

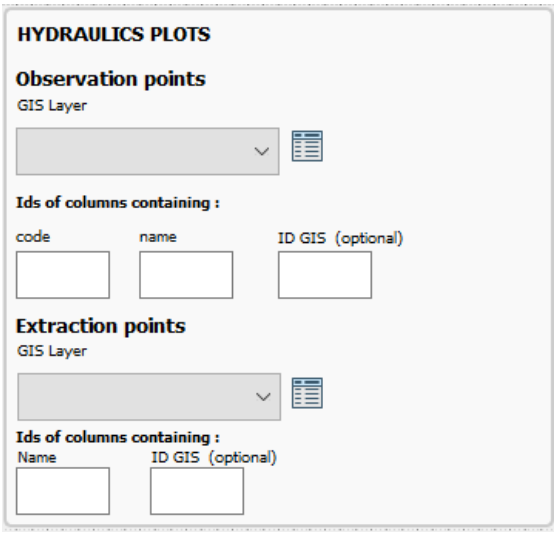


Figure 5.6: Configuration fields for "`HYD`" compartment extraction points

Extraction points of the "`AQ`" compartment

Extraction points

- "`name`": name of the observation point;
- "`ID LAYER`": identifier of the aquifer layer
- "`ID GIS`": GIS identifier of the observation point to which the observation point must be attached (optional field)

Observation points

- "`code`": identification code of the observation point, must be unique for each point. The observation file for the point must have the same name with the `.dat` extension;
- the "`name`" and "`ID GIS`" fields are also specified for the configuration of observation points

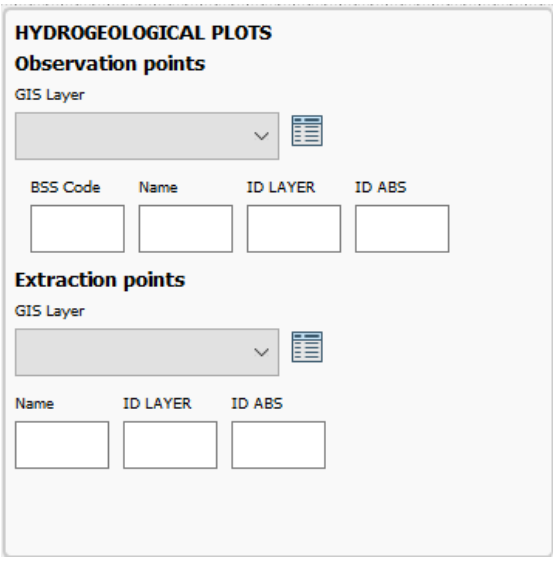


Figure 5.7: Configuration fields for "`AQ`" compartment extraction points

Extraction points must be contained in a `.shp` file different from the observation points. Points cannot be contained in the same file if they are extracted from two different compartments.

A preliminary save of the geometry configuration file can be saved by specifying a file path in the field located at the bottom left of the dialog box. This file (cf. Code 5.1) will be saved upon execution of the "**Settings**" dialog box if the configured project does not yet exist in the post-processing directory specified. Multiple project configurations can be saved in the same post-processing directory, provided they do not have the same project name.

5.3 Configuring Resolutions from a `.json` Configuration File

The geometric configuration of resolutions can be specified via a `.json` file according to the following structure:

```
{   // list of CaWaQS compartments to analyse (1 = AQ, 2 = HYD, 3 = WATBAL)
    "ids_compartment": [],

    "resolutionNames":
    {
        "1":
            [[""]], // list of names of aquifer meshes in the QGIS project
        "2" :
            [[""]], // name of the river mesh in the QGIS project
        "3" :
            [[""]] // name of the surface mesh in the QGIS project
        "4" :
            [[""]] // name of the unsaturated zone mesh in the QGIS project
        },
    "ids_col_cell": { // "compartment id" : "column number of cell id"
        "1" : "",  // "AQ id" : "absolute id column number"
        "2" : "",  //"HYD id" : "GIS id column number"
        "3" : ""   // "WATBAL id" : "cell id column number"
        },
    "obsNames": {
    // "compartment id" : "name of associated obs shp in the QGIS project"
        "1": "",
        "2" : ""
        },
    "obsIdsColCells": {
    // "compartment id" : "field number of the measurement point code"
        "1": "",
        "2": ""
        },
```

```
    "obsIdsColNames": {
    // "compartment id" : "field number of the measurement point name"
        "1": "",
        "2" : ""
        },
    "obsIdsColLayers": {
    // "compartment id" : "field number of the aquifer layer identifier"
        "1": "",
        "2" : null
        },
    "obsIdsCell": {
    // "comp. id" : "field number of the absolute or gis id of cell attached to obs"
        "1" : "",
        "2" : null
    },
     "extNames": {
    // "compartment id" : "name of associated extraction shp in the QGIS project"
        "2": "",
        "1": ""
    },
    "extIdsColNames": {
    // "compartment id" : "field number containing the extraction point name"
        "2": "",
        "1": ""
    },
    "extIdsColLayers": {
    // "compartment id" : "field number containing the aquifer layer identifier"
        "2": null,
        "1": ""
    },
    "extIdsColCells": {
    // "compartment id" : "field number containing the cell identifier"
        "1": "",
        "2": "",
    }
}
}
```

Code 5.1: Contents of a geometry configuration file

Part II

Developer Guide

CHAPTER 6

Key Concepts for Developers

Contents

6.1 Coupling Observation Points to Mesh Cells 46
6.2 The HYD Compartment: `ID_ABS` vs. `ID_GIS` 46

This chapter gathers a small set of **behavioural rules** that the code follows silently — decisions that are not obvious from reading a single function, but that a new developer must know before touching data flowing between CaWaQS and CaWaQS-Viz. It is the natural home for "why does the code do *that*?" knowledge; the architectural tour and the concrete editing recipes follow in chapter 7.

6.1 Coupling Observation Points to Mesh Cells

An observation point (a gauging station, a piezometer, ...) must be attached to exactly one mesh cell before its simulated values can be read. CaWaQS-Viz decides which cell to use in **two different ways**, and the choice is made *per observation*, not globally:

- **By identifier (preferred).** If the observation layer carries an explicit cell identifier — a configured `ID_ABS` column in the observation's attribute table (`.dbf`) — that identifier is used directly. This is the authoritative path: the coupling is exactly what the data declares, independent of geometry.
- **By geometry (fallback).** If no such identifier is provided, the code falls back to a **nearest-cell** search: it takes the observation's point geometry and finds the closest cell of the relevant mesh layer through a spatial-index nearest-neighbour lookup.



The two paths are **not** equivalent. The geometric fallback is a convenience for data that lacks explicit identifiers; it can attach a point to the wrong cell near mesh boundaries or where cells are small. Whenever authoritative results matter, provide `ID_ABS` on the observations and rely on the identifier path — the nearest-cell search should be treated as a last resort, not the default.

6.2 The HYD Compartment: `ID_ABS` vs. `ID_GIS`

For every compartment *except one*, a cell keeps the **same identifier** on both sides of the tool: the ID used inside CaWaQS is the ID used inside CaWaQS-Viz. This lets most of the code move IDs across the CaWaQS / CaWaQS-Viz seam without any conversion.

The **HYD (hydrological/river) compartment is the sole exception.** Its cells are identified differently on each side:

- inside CaWaQS, a HYD cell is identified by its `ID_ABS` (the absolute/internal CaWaQS index);
- inside CaWaQS-Viz (the GIS world), the same cell is identified by its `ID_GIS`.

The two are bridged by a **HYD correspondence table** produced with the simulation outputs, which pairs each `ID_GIS` with its `ID_ABS`. The translation is applied **in both directions**, at the two ends of the data path:

- **On input** (reading a HYD observation / feeding CaWaQS-Viz): the incoming CaWaQS-Viz-side `ID_GIS` is looked up in the correspondence table and **translated to** `ID_ABS` so that CaWaQS results can be addressed.
- **On output** (reporting HYD results back to the GIS): the reverse translation is applied, mapping `ID_ABS` back to `ID_GIS`.



Practical consequence: if you are debugging a HYD result and the IDs "don't line up", check whether you are comparing an **ID_ABS** (CaWaQS side) against an **ID_GIS** (CaWaQS-Viz side). For every other compartment the identifiers coincide and no such lookup happens; only HYD carries — and requires — this correspondence table.

CHAPTER 7

Architecture

Contents

7.1 Split Architecture: HTAS and CWV 49
7.2 The Three Layers of the Backend 49
7.3 Example Workflow: the Budget Bar Plot 52
7.4 Entry Gates 53
7.4.1 Client gate — L1 - HT CLIENT - MACRO 53
7.4.2 Developer gate — L2 - HT DEVELOPER - MICRO 53

This chapter describes the conceptual architecture — the HTAS/CWV split and the three backend layers — together with the **rules** a developer must follow when working within that structure, and the two entry gates through which all frontend → backend calls must pass. The concrete recipes for extending the application and the supporting tooling follow in the next chapters.

7.1 Split Architecture: HTAS and CWV

CaWaQS-Viz is built on top of a standalone scientific backend and consumes it through a well-defined public interface. Understanding the code therefore means understanding two things: how the QGIS application and the backend are kept *separate*, and how the backend is internally organised into *three layers*. This section describes both, and states the strict rules a developer must follow when working within this structure.

CaWaQS-Viz is maintained as two packages that are kept **strictly independent**:

- **`HydrologicalTwinAlphaSeries` (HTAS)** — the standalone scientific backend: simulation processing, domain structures (compartments, meshes, observations), configuration models, and the `HydrologicalTwin` facade. It lives under `external/HydrologicalTwinAlphaSeries`; in the published setup this path is a GitLab submodule pointing to the standalone backend repository.
- **`cawaqsviz` (CWV)** — the standalone QGIS application: dialog boxes, visualisation workflows, GIS integration, and application-side adapters.

Independence is a hard architectural **invariant**, not a style preference:

- HTAS must **never** import `qgis.*`, `PyQt5`, `processing`, or anything from `cawaqsviz/`. If you are about to add such an import inside `external/HydrologicalTwinAlphaSeries/`, stop — the work belongs on the CWV side;
- CaWaQS-Viz must consume HTAS only through its public API — the two entry gates (Sec. 7.4) — and keep QGIS-specific adaptation code on the application side. Runtime HTAS imports belong in `interface/`; `frontend/` modules access backend behaviour only through the application gateway;
- the backend is designed to be **usable from a notebook today and a server tomorrow** (Sec. 4.1). Anything that breaks notebook-from-pure-Python usage also breaks the future server;
- synchronisation between the two repositories happens through version control and dependency pinning, not through code duplication.

7.2 The Three Layers of the Backend

HTAS is organised as a **three-level architecture** in which each level may call the level below it, never the level above. Each level has one canonical entry module, and imports point **strictly downward** (L1 → L2 → L3): never up, never sideways within a level. Because every dependency points downward, the layers can never form an import loop — the dependency graph is acyclic.

```
L1 - HT CLIENT - MACRO        ht/client/hydrological_twin_client.py       (may import L2, L3)
L2 - HT DEVELOPER - MICRO     ht/developer/hydrological_twin_developer.py (may import L3)
L3 - SERVICES - ELEMENTARIES  services/                                   (imports nothing in ht/)
```

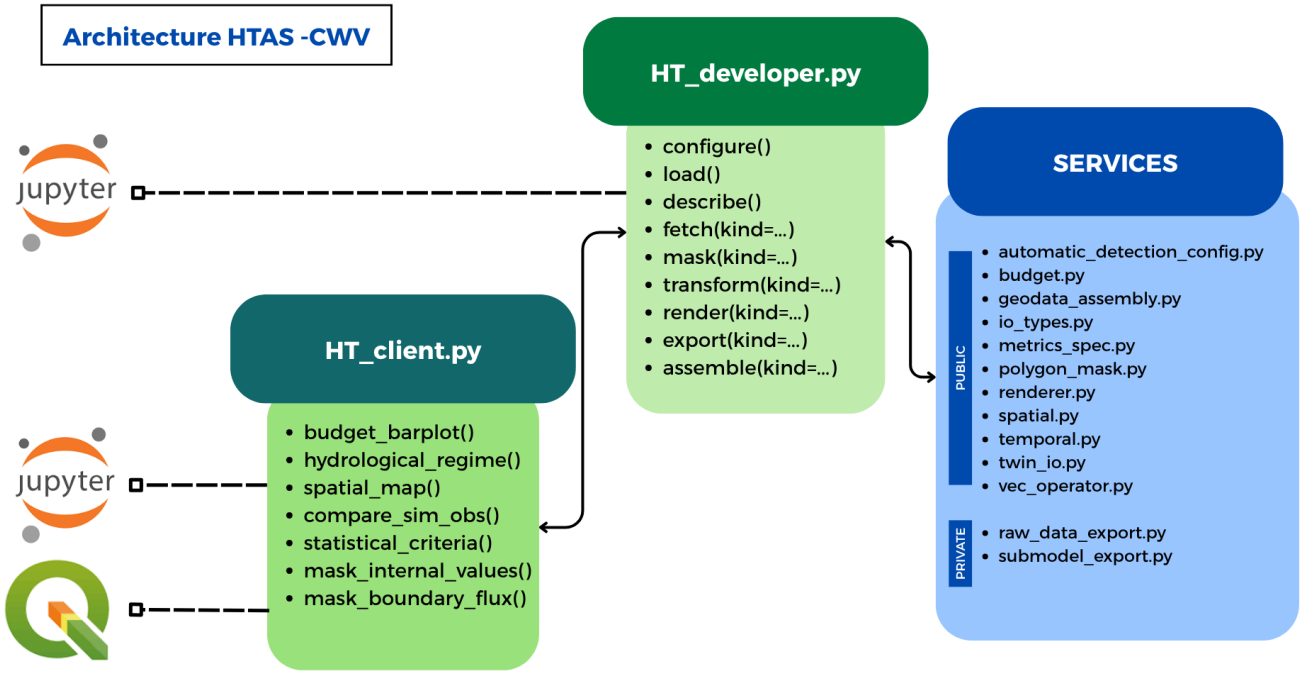


Figure 7.1: CaWaQS-Viz + HTAS architecture. The three backend layers are L1 (`HT_Client`), L2 (`HT_developer`), and L3 (`Services`).

Code 7.1: The three backend layers and their canonical entry modules.
Taken together with the QGIS-side layers (frontend and interface), the roles are summarised in Tab. 7.1.

Layer	Role
Frontend `frontend/`	QGIS dialog boxes. They own user interaction and route a single coarse call per workflow to the interface. `Qt/PyQt5` lives here and only here.
Interface `interface/`	`ExploreData` is the **only** gateway to the backend; `GeoDataProvider` turns QGIS vector layers into `GeoDataFrame`s so the backend stays QGIS-agnostic.
L1 - HT CLIENT - MACRO `ht/client/`	`HydrologicalTwinClient` exposes one method per user-facing dialog operation. Each delegates to a matching `operations_client.run_*` that owns the full `fetch` → `transform` → `render` chain and returns a typed `*Result` dataclass. Zero `qgis/PyQt5` imports — usable from a notebook today, from an HTTP server tomorrow.
L2 - HT DEVELOPER - MICRO `ht/developer/`	`HydrologicalTwin` is the micro-verb facade exposing 8 canonical verbs with an explicit `EMPTY` → `CONFIGURED` → `LOADED` → `READY` state machine. Useful for notebook exploration and for operations not yet promoted to the client.
L3 - SERVICES - ELEMENTARIES `services/`	Finest-grained computations and I/O: operators, renderer, twin-state reads and disk-cache provisioning (`twin_io.py`), and leaf data DTOs (`io_types.py`). No module under `services/` imports from `ht/` — the import graph is a strictly downward, acyclic DAG.

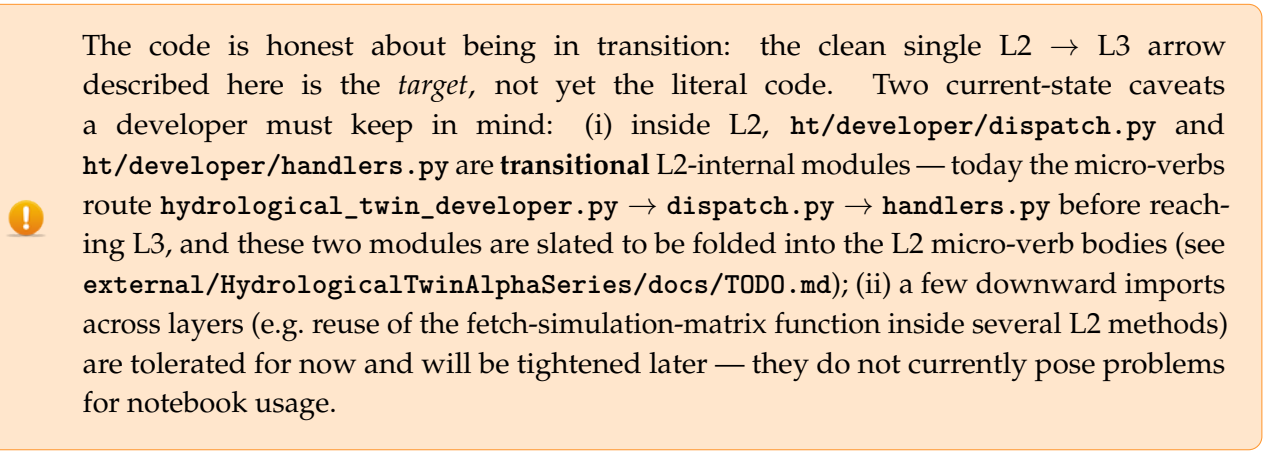
Table 7.1: Roles of the frontend, interface, and the three backend layers.

`ExploreData` calls **the client** for coarse operations (the normal path) and can still reach the developer twin *via* `client._twin` for low-level micro-verb calls. A more annotated view of the data flow is given in Fig. 7.1.

Golden rule — imports point downward only. Never up, never sideways within a level. No module under `services/` may import from `ht/client/` or `ht/developer/`. Shared data DTOs live at the L3 leaf (`services/public/io_types.py`) and are re-exported by the L2 `api_types.py`, so the edge stays downward. Because every dependency points downward, the graph can never form an import loop — it is a strictly downward, acyclic DAG.

Golden rule (corollary) — L1 only orchestrates, it does not handle data. L1 (`hydrological_twin_client.py` and, behind it, `operations_client.py`) is *pure orchestration*: it may only *call* L2 micro-verbs and *bundle* their typed results into a `*Result` dataclass. It must contain **no data handling of its own** — no `DataFrame` construction, no array reshaping or aggregation, no unit/path/provenance shaping, no disk I/O, no domain computation. Any such logic belongs *down* in an L2 micro-verb (`fetch/transform/render/export`, or a

new `kind=`) or in L3 `services/`. The test: if a line in `ht/client/` does anything other than call a lower layer or pack/unpack a typed result, it is in the wrong layer.



The code is honest about being in transition: the clean single L2 → L3 arrow described here is the *target*, not yet the literal code. Two current-state caveats a developer must keep in mind: (i) inside L2, `ht/developer/dispatch.py` and `ht/developer/handlers.py` are **transitional** L2-internal modules — today the micro-verbs route `hydrological_twin_developer.py` → `dispatch.py` → `handlers.py` before reaching L3, and these two modules are slated to be folded into the L2 micro-verb bodies (see `external/HydrologicalTwinAlphaSeries/docs/TODO.md`); (ii) a few downward imports across layers (e.g. reuse of the fetch-simulation-matrix function inside several L2 methods) are tolerated for now and will be tightened later — they do not currently pose problems for notebook usage.

7.3 Example Workflow: the Budget Bar Plot

The following walk-through shows what happens, step by step, when a user creates a water budget bar plot. It is one of the most complete workflows, chaining `fetch` → `transform` → `render`, and it is useful for understanding the number of round-trips between layers — especially when designing the future server API.

1. In the `DialogBudgetbarplot` dialog box, the user selects variables, a period, a frequency, and an aggregation.
2. The dialog issues a *single* coarse call to `ExploreData`, which forwards it to `HydrologicalTwinClient.budget_barp` which in turn calls `operations_client.run_budget_barplot`.
3. For each variable (rain, ETR, infiltration, runoff), the orchestration calls `twin.fetch` (reads the simulation matrix from disk through the services layer) then `twin.transform` (spatial mean + temporal aggregation).
4. A final `twin.render` call produces the plot; the renderer writes a PNG and a CSV to disk and returns their paths.
5. The paths are bundled into a `BudgetBarplotResult` dataclass and returned up the chain to the dialog, which opens the plot file.

What this reveals for API design:

- today, everything between `ExploreData` and the developer twin is **in-process**. The natural network seam is the `ExploreData` → `Client` boundary, not `ExploreData` → `HydrologicalTwin`;
- **one coarse operation = one server round-trip**. The dialog calls `client.budget_barplot(...)` once; the inner loop of 4 fetches + 4 transforms + 1 render stays server-side inside `run_budget_barplot`. The former "9-calls-per-plot" cost collapses to a single call at the network boundary;

- **the client *is* the future HTTP surface.** Each `HydrologicalTwinClient` method maps cleanly to one endpoint, with a typed `*Result` dataclass as the response schema. The developer micro-verb API stays available for notebook-style exploration;
- **rendering still writes to disk.** When the client moves server-side, render outputs will need to return data (or signed URLs) instead of local file paths — the `*Result` dataclasses are the right place to evolve that contract.

7.4 Entry Gates

All frontend → backend calls go through one of two surfaces. Do not reach around them.

7.4.1 Client gate — L1 - HT CLIENT - MACRO

`HydrologicalTwinClient` (`ht/client/hydrological_twin_client.py`) exposes **one method per user-facing dialog operation**:

- `budget_barplot`, `hydrological_regime`;
- `spatial_map_watbal`, `spatial_map_aq`;
- `compare_sim_obs`, `statistical_criteria`;
- `mask_watbal`, `mask_hyd_boundary`, `mask_aq_boundary`.

Plus the lifecycle methods `configure`, `load`, `describe`, `is_ready`, and the one-shot `build` classmethod. Each operation wraps a full `fetch` → `transform` → `render` chain server-side and returns a typed `*Result` dataclass (defined in `ht/client/api_types.py`, orchestrated by `ht/client/operations_client.py`). **This is the surface a future HTTP server will expose:** one coarse operation = one round-trip. Frontend dialogs should prefer this gate.

7.4.2 Developer gate — L2 - HT DEVELOPER - MICRO

`HydrologicalTwin` (`ht/developer/hydrological_twin_developer.py`) exposes the 8 canonical verbs:

- lifecycle (inline): `configure`, `load`, `describe`, `export`;
- dispatching (routed by `kind=` through the transitional `dispatch.py` → `handlers.py`, then down to L3): `fetch`, `mask`, `transform`, `render`.

State machine: `EMPTY` → `CONFIGURED` → `LOADED` → `READY`. Each verb checks the state via `_require_state` and raises `InvalidStateError` if called too early. The developer gate is the **power-user / notebook / exploration** surface. Use it for operations not yet promoted to the client, and for low-level work; `ExploreData` can still reach it *via* `client._twin` when needed.

CHAPTER 8

Development Workflow

Contents

8.1 Adding a New Dialog Operation 54
8.2 When Editing — Checklist 54
8.3 Conventions 55
8.4 Version Update 55
8.5 Code Documentation 56

This chapter gathers the **rules and processes** a developer must follow when working on CaWaQS-Viz: how to extend the application with a new operation, the checklist to run through before committing, the project conventions, and how versioning and code documentation are handled.

8.1 Adding a New Dialog Operation

To add a new dialog-level workflow, follow three steps:

1. define a `*Result` dataclass in `ht/client/api_types.py`;
2. implement `run_<name>` in `ht/client/operations_client.py` — it owns the `fetch` → `transform` → `render` orchestration;
3. add a one-line facade method on `HydrologicalTwinClient`, then call it from `ExploreData`.

Method bodies on the client class are intentionally thin ($\leq$ 5 statements). Keep them that way.

8.2 When Editing — Checklist

- touching dialog code? → `frontend/Dialog*.py`. Route through `ExploreData`;
- adding a new dialog-level workflow? → the three-step recipe above (Sec. 8.1);

- adding a new compute primitive? → `ht/developer/handlers.py` (compute) or `services/public/twin_io.py` (L3 state reads / disk I/O); route *via* `dispatch.py` with a new `kind=`;
- found a `qgis` / `PyQt5` import in `external/HydrologicalTwinAlphaSeries/`? → bug. Move it to `interface/` or `frontend/`;
- found a module under `services/` importing from `ht/client/` or `ht/developer/`? → bug. That is an *upward* import; imports must point downward only;
- found a frontend module reaching past `ExploreData` into HTAS internals? → bug. Route through the client gate;
- found data handling inside `ht/client/` (`DataFrame` building, array reshaping/aggregation, unit/path shaping, disk I/O, domain math)? → bug. L1 only orchestrates.

8.3 Conventions

- **Environment:** `pixi`-managed (`pixi.toml`, `pixi.lock`). QGIS, Python, and all dependencies are pinned in a single reproducible environment (Sec. 1.1);
- **Sync helpers:** `./verifState.sh` (diagnostics) and `./updateProject.sh` (ordered multi-repository / submodule sync);
- **Pre-lifecycle discovery** (no twin required): `HydrologicalTwinClient.detect_from_out_caw(...)` and `detect_project_neighbors(...)` — static helpers used by `DialogSettings` to auto-fill paths before `configure` / `load`;
- **Compatibility shim:** `cawaqsviz/backend`, if present, re-exports HTAS symbols. The real implementation lives only in `external/HydrologicalTwinAlphaSeries/`.

8.4 Version Update

The plugin version is specified in the `metadata.txt` file contained in code excerpt 8.1.

```
[general]
name=CaWaQS-Viz
qgisMinimumVersion=3.0
description=Description
version=0.1.16
author=Lise-Marie GIROD (Mines Paris - PSL), Nicolas Flipo (Mines Paris - PSL)
email=lise-marie.girod@minesparis.psl.eu

about=Python QGIS-based vizualization software for CaWaQS3.x

repository=https://gitlab.com/gviz/cawaqsviz
```

Code 8.1: Mandatory information to fill in the `metadata.txt` file

8.5 Code Documentation

The code documentation is published on the following link via GitLab Pages: https://cawaqsviz-gviz-adf347ff6d9d45c174db091497a2dfc0f14305cdb4ed9070.gitlab.io/. The documentation is maintained through docstrings in the application modules, following the formatting shown in the example below.

```python
class Exemple:
    """
    Resume des fonctionnalites de la classe.
    """

    def __init__(self, arg1):
        """
        Methode constructeur.

        :param arg1: descriptif du parametre
        :type arg1: str
        """
        self.arg1 = arg1

    def do_something(self, arg1: list) -> str:
        """
        Descriptif de la fonction.

        :param arg1: descriptif du parametre
        :type arg1: list
        :return: descriptif de ce que la fonction retourne
        :rtype: str
        """
        return ", ".join(map(str, arg1))
```

Code 8.2: Docstring formatting for automatic integration into code documentation

Compilation is performed by `Sphinx`[1]. The `docs/sources` directory contains the configuration files (`Conf.py`), as well as the documentation content. When integrating new modules into the plugin, adding them to the index (`index.rst`) is necessary for the code docstring to appear on the documentation page. This index integrates 3 files: `frontend.rst`, `interface.rst`, `backend.rst`, which contain the classes to be documented. Integrating a new module into the application requires it to be added to these files in the format of code excerpt 8.3. Updates to the documentation source files must be committed and synchronised with the repositories.

```
Module
------

.. automodule:: cawaqsviz.Module
   :members:
   :show-inheritance:
```

[1] whose documentation is accessible via https://www.sphinx-doc.org/en/master/

```
   :undoc-members:
```

Code 8.3: Docstring formatting for automatic integration into code documentation

Finally, with each new commit to the `Main` or `Beta` branches, the documentation is compiled according to the content of the `docs/source` directory of the repository.

```
image: python:3.9

stages:
  - build
  - deploy

before_script:
  - pip install -U pip
  - pip install -r requirements.txt

build-docs:
  stage: build
  script:
    - sphinx-build -b html docs/source public
  artifacts:
    paths:
      - public

pages:
  stage: deploy
  script:
    - echo "Pages deployed"
  artifacts:
    paths:
      - public
  only:
    - main
    - beta
```

CHAPTER 9

Developer Tooling

Contents

9.1 Debugging Tools 59

9.1.1 The `Plugin Reloader` plugin 59

9.1.2 The QGIS plugin `First Aid` 59

9.1.3 VS-Code 60

9.2 Creating Dialog Boxes with `Qt Designer` 64

This chapter covers the **optional tools** that make day-to-day development easier: the debugging utilities (plugin reloading, error inspection, and a full VS-Code debugger attached to QGIS), and `Qt Designer` for authoring dialog boxes.

9.1 Debugging Tools

9.1.1 The `Plugin Reloader` plugin

This utility allows reloading a plugin after a code modification. Thus it is not necessary to reload QGIS after each code change.

The plugin can be installed from the `QGIS` extension manager.



Figure 9.1: Description sheet for `Plugin Reloader`

9.1.2 The QGIS plugin `First Aid`

The `First Aid` plugin allows investigating errors occurring during plugin execution.

9.1.3 VS-Code

The **debugvs** plugin allows connecting a **VS-Code** debugging instance to the plugin execution from QGIS. This debugging configuration allows using all VS Code features during debugging (breakpoints, watch, call stack...). Setting up this debugging configuration is done as follows:

1. install **debugpy** with the command **!pip install debugpy** from the QGIS Python command prompt;
2. if the plugin development directory is not located in `C://Users/`*USER*`/AppData//QGIS/QGIS3/profiles/defaul` create a symbolic link in this directory pointing to the development directory.
3. install the **debugvs** plugin by downloading its script from the GitLab repository https://github.com/lmotta/debug_vs and activate the plugin by importing the **.zip** folder in **QGIS** (cf. Fig. 9.2);

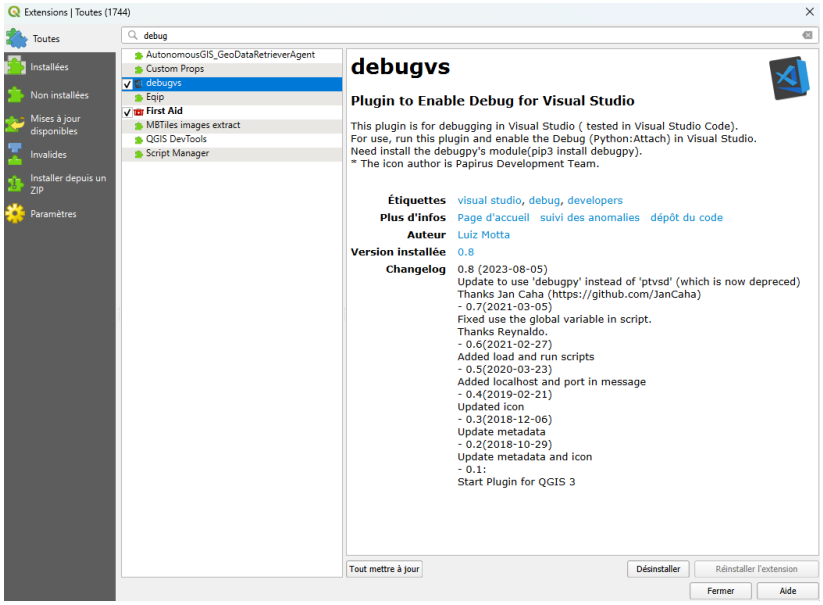


Figure 9.2: `debugvs` plugin

4. configure VS Code to connect to QGIS during a debugging session:
   (a) in the CaWaQS-Viz development directory, create a **.vscode** sub-directory (add this directory to the **.gitignore** file);
   (b) create a **settings.json** file in the **.vscode** sub-directory and copy the content from code 9.1. Optionally modify the directory paths to match your QGIS and Python installation and operating system. This one is for Windows.

```
1  {
2
3      "workbench.colorCustomizations": {
4          "titleBar.activeBackground": "#006823",
5          "editor.background": "#000000"
```

```
        },

        "python.linting.pylintEnabled": true,
        "python.jediEnabled": true,


        "python.pythonPath": "C:/OSGeo4W64/bin/python-qgis.bat",


        "python.linting.pylintArgs": [
            "--extension-pkg-whitelist=PyQt5,qgis,db_manager",
            "--disable=all",
            "--enable=F,E,unreachable,duplicate-key,unnecessary-semicolon,
global-variable-not-assigned,unused-variable,binary-op-exception,bad-format-
string,anomalous-backslash-in-string,bad-open-mode"
        ],


        "terminal.integrated.env.windows": {

            "PATH": "C:\\OSGeo4W\\apps\\qgis\\bin;C:\\OSGeo4W\\apps\\
Python312;C:\\OSGeo4W\\apps\\Python312\\Scripts;C:\\OSGeo4W\\apps\\Qt5\\bin;C
:\\OSGeo4W\\apps\\Python312\\Scripts;C:\\OSGeo4W\\bin;C:\\Windows\\System32;C
:\\Windows;C:\\Windows\\system32\\WBem;C:\\OSGeo4W\\apps\\Python312\\lib\\
site-packages\\pywin32_system32;C:\\OSGeo4W\\apps\\Python312\\lib\\site-
packages\\numpy\\.libs",

            "PYTHONHOME": "C:\\OSGeo4W\\apps\\Python312",
            "PYTHONPATH": "C:\\OSGeo4W\\apps\\qgis\\python;%PYTHONPATH%",

            "GDAL_DATA": "C:\\OSGeo4W\\share\\gdal",
            "GDAL_DRIVER_PATH": "C:\\OSGeo4W\\bin\\gdalplugins",
            "GDAL_FILENAME_IS_UTF8": "YES",

            "GEOTIFF_CSV": "C:\\OSGeo4W\\share\\epsg_csv",

            "O4W_QT_BINARIES": "C:/OSGeo4W/apps/Qt5/bin",
            "O4W_QT_DOC": "C:/OSGeo4W/apps/Qt5/doc",
            "O4W_QT_HEADERS": "C:/OSGeo4W/apps/Qt5/include",
            "O4W_QT_LIBRARIES": "C:/OSGeo4W/apps/Qt5/lib",
            "O4W_QT_PLUGINS": "C:/OSGeo4W/apps/Qt5/plugins",
            "O4W_QT_PREFIX": "C:/OSGeo4W/apps/Qt5",
            "O4W_QT_TRANSLATIONS": "C:/OSGeo4W/apps/Qt5/translations",
            "QT_PLUGIN_PATH": "C:\\OSGeo4W\\apps\\qgis\\qtplugins;C:\\OSGeo4W
\\apps\\Qt5\\plugins",

            "QGIS_PREFIX_PATH": "C:/OSGeo4W/apps/qgis",

            "VSI_CACHE": "TRUE",
            "VSI_CACHE_SIZE": "1000000"
        },
```

```
            "python.languageServer": "Jedi",
        }
```

Code 9.1: Workspace configuration file. This file defines the Visual Studio Code workspace configuration for QGIS plugin development. It adapts the VS Code Python environment to that used by QGIS (installed via OSGeo4W).

(c) create a `launch.json` file in the `.vscode` sub-directory, copy the content from code 9.2, modify the `localRoot` and `remoteRoot` paths to match your workspace configuration.

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Attach QGIS Debug",
            "type": "python",
            "request": "attach",
            "connect": {
                "host": "localhost",
                "port": 5678
            },
            "pathMappings": [
                {
                    "localRoot": "C:\\Program Files\\$QGIS$\\apps\\qgis-ltr\\
    python\\plugins\\cawaqsviz",
                    "remoteRoot": "C:\\Users\\$USER$\\AppData\\Roaming\\QGIS\\
    QGIS3\\profiles\\default\\python\\cawaqsviz"
                }
            ],
            "justMyCode": false
        }
    ]
}
```

Code 9.2: VS-Code debugger configuration file. In this configuration, the "connect" section indicates the address and port on which QGIS exposes the debugging session (here localhost:5678). The "pathMappings" block establishes the correspondence between the local project paths open in VS Code (localRoot) and the paths used by QGIS to execute the plugin (remoteRoot). This correspondence is essential for breakpoints placed in VS Code to be correctly associated with the files actually executed by QGIS.

5. Establish the QGIS/VS-Code connection

(a) authorise the debugger connection to QGIS via the `debugvs` plugin (cf. Fig. 9.3);

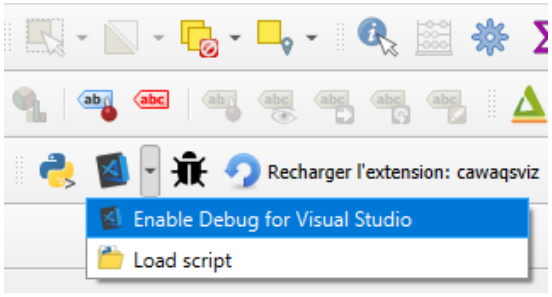


Figure 9.3: Establishing a QGIS/VS-Code connection from the `debugvs` plugin

(b) launch the VS-Code debugging session (cf. Fig. 9.4).

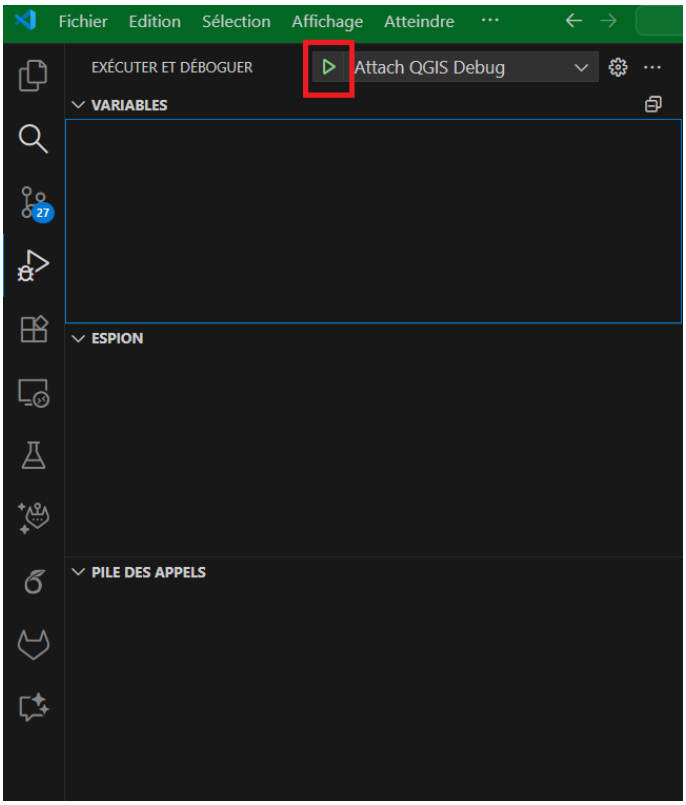


Figure 9.4: Launching a debugging session from VS-Code

The connection between the debugger and QGIS is terminated from VS-Code by clicking on `"Disconnected"` in the debugging toolbar (cf. Fig. 9.5)



Figure 9.5: VS-Code debugger toolbar

9.2 Creating Dialog Boxes with `Qt Designer`

The **`Qt Designer`** application, provided with the QGIS installation (installation with OSGEO), is a tool for creating dialog boxes in the form of .ui files. In the CaWaQS-Viz application, all dialog boxes have their own `.py` file to which the `.ui` file, of the same name, is associated with the dialog box's Python class.

```
# This loads your .ui file so that PyQt can populate your plugin with the elements from Qt
    Designer
FORM_CLASS, _ = uic.loadUiType(
    os.path.join(os.path.dirname(__file__), "DialogComparesimobs.ui")
)


class DialogComparesimobs(Dialog, FORM_CLASS):
    def __init__(self, iface:QgisInterface, log_file_path:str, parent=None):
        """Constructor Method

        :param iface: Qgis Interface
        :type iface: QgisInterface
        :param logFilePath: Path of the cawaqs-viz log file, defaults to None
        :type logFilePath: str, optional
        :param parent: Parent class, defaults to None
        :type parent: _type_, optional
        """
        super(DialogComparesimobs, self).__init__(parent)
        self.setupUi(self)
        self.iface = iface
        self.logFilePath = log_file_path
        self.simData = False
```

Code 9.3: Initialisation of the "Compare SIM/OBS" dialog box from a .ui file

Bibliography

Gupta, H. V., Clark, M. P., Vrugt, J. A., Abramowitz, G., and Ye, M. (2012). Towards a comprehensive assessment of model structural adequacy. *Water Resources Research*, 48(8). _eprint: https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2011WR011044.

Nash, J. E. and Sutcliffe, J. V. (1970). River flow forecasting through conceptual models part I — A discussion of principles. *Journal of Hydrology*, 10(3):282–290.

List of Figures

1 The CaWaQS-Viz ecosystem — position of the plug-in within the CaWaQS toolchain. vi

1.1 QGIS dialog box for plugin management. 6
1.2 Activating the CaWaQS-Viz extension in the QGIS application. 6

2.1 Home dialog box of the CaWaQS-Viz application. 10
2.2 The "`Settings`" dialog box of the plug-in, selecte the creation of a new project. 11

3.1 Calibration tools dialog box. 16
3.2 Dialog box for modifying parameterisation files. 17
3.3 Dialog box for linking an attribute table to its resolution. 18
3.4 "`Statistical Criteria`" dialog box. 21
3.5 Dialog box for calculating the run-off coefficient. 22
3.6 "`Compare sim/obs`" dialog window. 23
3.7 Spatialisation of mean inter-annual infiltration flux [mm] (a) and piezometry [$mNGF$] (b) simulated at the scale of the Loing hydrosystem 24
3.8 Example of a graphical output for the simulated multi-year monthly hydrological budget . . 25
3.9 "`Hydrological Regime`" dialog box for calculating multi-year monthly hydrological regimes. 26
3.10 Example of a graphical output for the simulated multi-year monthly hydrological regime . . 26
3.11 "`Extract Data from a Sub Area`" dialog box. 27
3.12 "`AQ boundary geopackage`". 30

5.1 WATBAL compartment configuration fields 39
5.2 HYD compartment configuration fields 39
5.3 NSAT compartment configuration fields 40
5.4 AQ compartment configuration fields 40
5.5 Window for selecting the field identifier containing the "`ID ABS`" in the attribute table in the case of heterogeneous field identifiers across layers 40

5.6 Configuration fields for "`HYD`" compartment extraction points 41
5.7 Configuration fields for "`AQ`" compartment extraction points 41

7.1 CaWaQS-Viz + HTAS architecture. The three backend layers are L1 (`HT_Client`), L2 (`HT_developer`), and L3 (`Services`). 50

9.1 Description sheet for `Plugin Reloader` 59
9.2 `debugvs` plugin 60
9.3 Establishing a QGIS/VS-Code connection from the `debugvs` plugin 63
9.4 Launching a debugging session from VS-Code 63
9.5 VS-Code debugger toolbar 63

List of Tables

1 Latest implementations in the application vii

3.1 Parameterisation files supported by `CaWaQS-Viz`. 18
3.2 Field names of output tables for each statistical criterion 20
3.3 Accepted `(compartment, outtype, param, unit)` tuples for **`mask_internal_values`**. . . . 31

5.1 CaWaQS resolutions that can be imported as compartment resolution. 38
5.2 Structure of GIS layers for extraction points and associated graphical outputs 38

7.1 Roles of the frontend, interface, and the three backend layers. 51

List of Codes

2.1 Expected layout of the CaWaQS output directory. 12
2.2 Expected layout around the QGIS project file. 13
3.1 Example of `specs` for `mask_internal_values`. 31
3.2 Example call of `mask_aq_boundary`. 32
5.1 Contents of a geometry configuration file 42
7.1 The three backend layers and their canonical entry modules. 49
8.1 Mandatory information to fill in the `metadata.txt` file 55
8.2 Docstring formatting for automatic integration into code documentation 56
8.3 Docstring formatting for automatic integration into code documentation 56
9.1 Workspace configuration file. This file defines the Visual Studio Code workspace configuration for QGIS plugin development. It adapts the VS Code Python environment to that used by QGIS (installed via OSGeo4W). 60
9.2 VS-Code debugger configuration file. In this configuration, the "connect" section indicates the address and port on which QGIS exposes the debugging session (here localhost:5678). The "pathMappings" block establishes the correspondence between the local project paths open in VS Code (localRoot) and the paths used by QGIS to execute the plugin (remoteRoot). This correspondence is essential for breakpoints placed in VS Code to be correctly associated with the files actually executed by QGIS. 62
9.3 Initialisation of the "Compare SIM/OBS" dialog box from a .ui file 64